

审定成绩： A

重庆邮电大学
毕业设计（论文）

中文题目	面向静态稀疏 Transformer 的注意力计算加速器的研究、设计与实现
英文题目	Research, Design and Implementation of an Attention Computation Accelerator for Static Sparse Transformers
学院名称	计算机科学与技术学院 (示范性软件学院)
学生姓名	蒋佳鑫
专 业	计算机科学与技术
班 级	04012204
学 号	2022212045
指导教师	陈鹏 副教授
答 辩 组 负 责 人	沈靖程 讲师

二〇二六年五月
重庆邮电大学教务处制

计算机科学与技术学院本科毕业设计(论文)

诚信承诺书

本人郑重承诺：

我向学院呈交的论文《面向静态稀疏 Transformer 的注意力计算加速器的研究、设计与实现》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级 2022 级

专业 计算机科学与技术

班级 04012204

承诺人签名 蒋佳鑫

2026 年 5 月 18 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

学生签名：蒋佳鑫

日期：2026 年 5 月 18 日

指导老师签名：孙明

日期：2026 年 5 月 18 日

摘要

Transformer 模型凭借其卓越的全局特征提取能力，在自然语言处理、计算机视觉及多模态等领域取得了革命性突破。然而，随着应用场景向长文档理解等长序列任务拓展，其核心组件自注意力机制的计算复杂度呈二次方增长，注意力模块开销可占总量的 47.1%至 93.4%，成为资源受限设备上的部署瓶颈。为缓解这一压力，稀疏注意力被引入，通过仅计算部分高相关性查询-键对将复杂度降至近线性。然而，现有加速器在处理融合滑动窗口、全局及随机等模式的混合静态稀疏注意力时，普遍存在数据流僵化、资源利用率不均等问题，缺乏足够灵活性。

针对上述问题，本文提出面向混合静态稀疏注意力模式的可重构加速器 ReSA。首先，本文分析了混合稀疏注意力的数据重用特性，揭示了非局部与局部注意力数据流强耦合导致的资源闲置瓶颈，并提出数据分割与重排技术，将非规则稀疏模式转换为规则的结构化数据流，结合后归一化方法保证计算等价性。其次，本文设计了 ReSA 的硬件框架，将混合稀疏计算划分为局部、水平全局、垂直全局及随机四种模式，通过引入可重构条打破传统处理单元阵列的僵化互连约束，通过增强型权重求和模块实现加权累加与直接累加的无缝切换。在此基础上，处理单元采用四级流水线结构，结合基于查找表的分段线性近似优化 Softmax 计算。

实验结果表明，与最先进的 SALO2 加速器相比，ReSA 在混合稀疏注意力模式下实现了更高资源利用率，处理单元总数减少三分之一。在长序列任务中，ReSA 实现了 1.28 至 1.52 倍性能加速及 0.93 至 1.14 倍相对能效，验证了该设计的有效性与优越性。

关键词：Transformer 模型，稀疏注意力，硬件加速器，可重构架构，混合稀疏模式

Abstract

Transformer models have achieved revolutionary breakthroughs in natural language processing, computer vision, and multimodal learning due to their exceptional global feature extraction capabilities. However, as application scenarios expand to long-sequence tasks such as long document understanding, the computational complexity of their core component, the self-attention mechanism, grows quadratically. The attention module can account for 47.1% to 93.4% of the total computational overhead, becoming a deployment bottleneck on resource-constrained devices. To alleviate this pressure, sparse attention has been introduced, reducing the complexity to nearly linear by computing only a subset of highly relevant query-key pairs. Nevertheless, existing accelerators commonly suffer from rigid dataflows and imbalanced resource utilization when processing hybrid static sparse attention that integrates sliding window, global, and random patterns, lacking sufficient flexibility.

To address these issues, this thesis proposes ReSA, a reconfigurable accelerator for hybrid static sparse attention patterns. First, we analyze the data reuse characteristics of hybrid sparse attention, revealing the resource idling bottleneck caused by the tight coupling between non-local and local attention dataflows. We then propose data partitioning and rearrangement techniques to convert irregular sparse patterns into regular structured dataflows, combined with a post-normalization method to ensure computational equivalence. Second, we design the hardware framework of ReSA, which partitions hybrid sparse computations into four modes: local, horizontal global, vertical global, and random attention. By introducing a reconfigurable bar, the rigid interconnection constraints of traditional processing element arrays are broken. An enhanced weight summation module enables seamless switching between weighted accumulation and direct element-wise accumulation. On this basis, the processing unit adopts a four-stage pipeline structure, incorporating a lookup table-based piecewise linear approximation to optimize Softmax computation.

Experimental results demonstrate that compared with the state-of-the-art SALO2 accelerator, ReSA achieves higher resource utilization under hybrid sparse attention patterns while reducing the total number of processing units by one-third. On long-sequence tasks, ReSA achieves a performance speedup of $1.28\times$ to $1.52\times$ and a relative

energy efficiency of 0.93 to 1.14, validating the effectiveness and superiority of the proposed design.

Keywords: Transformer Model, Sparse Attention, Hardware Accelerator, Reconfigurable Architecture, Hybrid Sparse Patterns

目录

第 1 章 引言	1
1.1 研究背景和意义	1
1.2 国内外研究现状	2
1.2.1 国外研究现状	3
1.2.2 国内研究现状	4
1.3 主要内容和工作安排	5
第 2 章 背景和相关工作	7
2.1 标准注意力机制与性能分析	7
2.2 稀疏注意力机制	10
2.3 相关工作	12
2.4 本章小结	15
第 3 章 混合稀疏注意力的数据流特征分析与调度	17
3.1 数据流与数据重用特性分析	18
3.2 数据分割与重排技术	19
3.2.1 后归一化方法的数学推导	20
3.3 本章小结	21
第 4 章 基于可重构条与权重求和模块的解耦架构	23
4.1 整体框架概述	24
4.2 核心硬件架构	25
4.2.1 局部注意力模式的生命周期	25
4.2.2 水平全局注意力模式的完整生命周期	26
4.2.3 垂直全局注意力模式的完整生命周期	27
4.2.4 随机注意力模式的完整生命周期	28
4.3 本章小结	29
第 5 章 硬件设计	31
5.1 稀疏注意力加速器架构与数据布局	31

5.1.1 RCBa内部结构设计.....	32
5.2 PE 设计	34
5.2.1 Stage1: 点积计算与动态路由.....	35
5.2.2 Stage2: 非线性指数运算与 LUT 近似.....	35
5.2.3 Stage3: 对齐累加与 Softmax 分母计算	36
5.2.4 Stage4: 加权求与输出.....	37
5.3 增强型 WSM 设计	39
5.4 本章小结.....	40
第 6 章 实验评估.....	41
6.1 实验设置.....	41
6.2 性能分析与对比.....	42
6.3 本章小结.....	45
第 7 章 总结与展望.....	47
7.1 主要研究贡献总结.....	47
7.1.1 数据流层面的解耦与重塑.....	47
7.1.2 微架构层面的可重构设计.....	48
7.1.3 硬件实现与严谨性验证.....	48
7.2 未来工作展望.....	49
7.2.1 动态稀疏剪枝支持.....	49
7.2.2 多 FPGA 分布式加速架构	49
7.2.3 算法—硬件协同设计优化.....	50
参考文献.....	53
致谢.....	57

第 1 章 引言

Transformer 模型凭借其自注意力机制在自然语言处理、计算机视觉等领域的卓越表现，已成为现代人工智能系统的核心技术之一。然而，随着应用场景向长序列任务不断拓展，自注意力机制的二次复杂度问题日益成为制约模型部署的关键瓶颈。为缓解这一压力，稀疏注意力机制被广泛研究，但现有硬件加速器在处理融合多种模式的混合静态稀疏注意力时，普遍存在灵活性不足与资源利用率低下的问题。针对上述挑战，本文提出一种面向混合静态稀疏注意力的可重构加速器 ReSA，从数据流调度与硬件微架构两个层面实现不同稀疏模式的计算解耦。本章首先阐述研究的背景与意义，继而综述国内外相关研究现状，最后介绍本文的主要内容和结构安排。

1.1 研究背景和意义

近年来，基于注意力机制的神经网络，如 Transformer^[1]及其衍生出来的模型，已经在自然语言处理（NLP）、计算机视觉（CV）还有多模态学习这些人工智能的领域里，展现出了革命性的优势。与传统的循环神经网络（RNN）^{[1][3]}和卷积神经网络（CNN）^[4]不同，自注意力机制是通过算序列中任意两个元素之间的相关性权重，来提取和整合整个全局上下文的信息，这样就能绕开序列依赖和局部感受野带来的限制，在处理很复杂的序列数据时，兼具效率与精度。然而，自注意力机制存在一个固有缺陷，就是它的计算复杂度会随着序列长度增加而呈现出二次方的增长（也就是 $O(n^2)$ ），这一固有特性在处理长上下文任务时，比如长文档理解、高分辨率图像生成，会变成一个挺严重的计算瓶颈。有研究就发现，在基于 Transformer 的长序列任务里面，注意力模块的计算量能占到总计算量的 47.1%到 93.4%^[5]，这不仅拖慢了模型的实时推理能力，也大大加重了硬件的能耗负担。

为了突破这个计算上的瓶颈，稀疏注意力机制应运而生。其的核心想法，就是引入一种特定的稀疏模式，只去算相关性高的查询-键对，这样就能把复杂度从二次方降到接近线性。常见的稀疏模式包括滑动窗口注意力、扩展窗口注意力、全局注意力，还有随机注意力等。不过，为了在保持模型精度的同时把硬件能效拉到最

高，现在这些稀疏 Transformer 模型（比如 Longformer^[6]、BigBird^[7]）往往不会只用单一的稀疏策略，而是把以上几种模式混在一起用。例如，BigBird 模型在同一个注意力层里面，同时使用了局部滑动窗口、全局 Token 之间的交互，以及随机采样连接。

虽然这种混合静态稀疏注意力机制在算法层面上已经取得了显著成功，但现有的硬件加速器在处理这种混合模式的时候，在灵活性方面面临严峻挑战。传统的稀疏加速器，或是很依赖软件层面的复杂分区调度，或就是在硬件里给不同的稀疏模式单独配上专用的处理元件（Processing Element，以下简称 PE）。此种紧耦合的设计在处理混合稀疏模式的时候，灵活性瓶颈尤为突出：当非局部的注意力（比如全局注意力或随机注意力）的 Token 密度增加，或者是它们的空间分布没有与局部注意力的移动窗口对齐的时候，原本的数据复用机制就会直接失效。加速器里的非局部组件只能当作独立任务来调度，这样专用的处理单元就会出现严重的资源空置，导致资源利用率低下、流水线停顿，以及硬件冗余的问题。

针对以上出现的局限，进行支持混合稀疏注意力的可重构加速器研究，是有理论价值和现实意义的。此研究，就是从硬件的最底层去打破对局部注意力数据流的刚性依赖，通过引入一种可重构的数据分布和收集机制，来实现对多种稀疏模式的解耦，使各类模式能够独立、高效地完成计算。这样不仅能在大幅减少硬件资源开销的同时（与现在最先进的方案比，大约能省掉三分之一的 PE 数量），还能保持很高的吞吐量，同时降低芯片面积与功耗，从而为边缘设备和端侧设备高效地部署长序列 Transformer 模型，提供关键的硬件支撑。

1.2 国内外研究现状

在 Transformer 硬件加速领域，针对稀疏注意力机制的研究已从早期单纯的计算量节省，逐步演变为硬件—软件协同优化的内存访问消除。现有的研究根据地域和学术团队的不同，展现出了从底层拓扑映射到上层模型架构协同设计的多样化探索路径。

1.2.1 国外研究现状

国外在稀疏注意力这块起步相对较早，早期主要做一些基础探索和底层 I/O 感知优化，并且经历了一个从动态稀疏到静态结构化稀疏的明显范式转变。

一方面是在早期阶段，研究主要聚焦于动态稀疏性。例如 SpAtten^[8] 此类加速器，本意为通过运行时的动态剪枝去减少计算量，但由此产生的随机稀疏模式引发了两个主要问题——一是内存访问变得很不规则，二是负载均衡也容易出现瓶颈，反而增加了额外的硬件开销。正是因为此动态预测带来的开销非常大，所以近年来的研究开始转向两个方向：一个是如何消除预测开销，另一个是如何有效利用稀疏模式里的几何局部性。具体而言，PADE^[9] 使用了一种“预测消除”的技术，其通过位串行启用融合机制，在执行期间能直接识别出无效的计算块，把传统预测器里多余的硬件给彻底绕过去；DOTA^[10] 方面，则提出了一个“弱注意力检测”的机制，能在硬件层面直接跳过对边际贡献不大的注意力块，从而实现全模型路径上的稀疏加速。在更细粒度的结构优化方面，DynaX^[11] 亦开展了大量相关研究工作，其探索了一种动态 X:M 的细粒度结构剪枝方法，能在保持模型精度的前提下，显著提升硬件并行效率。

另外一方面，随着长序列任务变得越来越普遍，如何把特定的稀疏模式高效地映射到硬件拓扑结构上，成了一个研究热点。FlashAttention^[12] 系列立住了 I/O 感知优化的理论基础，其核心贡献在于证明了：在内存受限的硬件里，减少内存访问比减少计算对性能的提升更有效，这也为后续的硬件数据流设计提供了理论支撑。在此基础上，ASADI^[13] 进一步挖掘了注意力机制里所蕴含的“对角线局部性”，通过对角线数据格式（即 DIA）和原位计算，能显著减少滑动窗口模式下面的随机访问开销。此外，为了解决不同模型之间稀疏策略差异太大而导致的硬件碎片化问题，FlashOmni^[14] 尝试着去搭一个统一的稀疏引擎，用灵活的“稀疏符号”去标准化各种稀疏策略，这样就能提升硬件在不同模型架构之间的通用性。从另一角度来看，尽管以上工作在某些特定的稀疏模式上表现优异，然一旦面临把全局、局部和随机机制都揉在一起的混合静态稀疏结构（例如 BigBird），现有的国外架构还是普遍存在资源利用率不均和流水线停顿的瓶颈。

1.2.2 国内研究现状

国内学术界，在 Transformer 底层加速的协同设计上亦取得了显著的进展，其研究视角已基本覆盖了从模型架构创新到硬件微架构优化的整条技术链路。

第一方面是在模型架构这个层面上，国内许多学者都在研究如何设计出更适合硬件实现的稀疏注意力范式。例如，清华大学提出的 StarTransformer^[15]，把滑动窗口和全局注意力给结合在一起；这样既能保证模型捕获到长距离依赖关系，又为底层的硬件设计出了更规则的稀疏拓扑结构，后续在做硬件映射的时候更为便于操作。

第二方面是在硬件微架构这个层面上，国内的一些团队在数据流重构和计算资源复用这方面也提出了若干有代表性的方案。例如中国科学院团队做的 Ayaka 加速器^[5]，其通过低秩估计和异构数据流，把注意力模块的计算复杂度有效降低，还能为不同维度的注意力计算提供定制化的数据通路。再论及静态稀疏注意力加速这个方向，SALO2 加速器^[16]基本是这个领域里一个重要的基线工作，其初步解决了静态稀疏模式的硬件映射问题，尤其是其提出的对角线重用数据流，在处理纯局部稀疏模式的时候，效率极高。

然而，以 SALO2 为代表的一些现有国内架构，在处理混合模式的时候仍存在一些显著的局限性。这类架构过于依赖软件层面完美的分区调度，一旦面临如 BigBird 这样的混合稀疏场景，则会出现问题——因为非局部的注意力（包括全局注意力和随机注意力）与局部注意力的数据流之间是强耦合关系，所以只要配置参数不是最优的，SALO2 就无法把计算任务高效地分配到 PE 上，导致硬件冗余变得很严重，资源利用率也跟着急剧降低。也正是因为这个原因，最近的一些研究（例如本文要介绍的 ReSA 架构）正试图打破这一僵局，其通过引入可重构总线 and 增强型权重求和模块，在硬件层面上解耦掉不同稀疏模式的计算依赖，这样即使把 PE 数量减少三分之一，也能在混合稀疏模式下实现很高的利用率。综合而言，如何设计出一种能原生支持混合模式、同时架构灵活性又高的加速器，依然是目前国内这个领域里一个亟待突破的关键问题。

1.3 主要内容和工作安排

本文主要研究的是混合静态稀疏注意力机制在硬件上的高效映射问题，并在此基础上提出了一种名为 **ReSA** 的可重构稀疏注意力加速器架构。整篇文章一共分成 7 个章节，具体的结构安排和内容有以下几个方面。

第 1 章是引言。这章会先论及 **Transformer** 里面的自注意力机制在长序列任务中遇到的计算瓶颈，接着梳理现有的一些稀疏注意力加速器在处理混合模式时存在的灵活性缺陷和资源冗余问题，进而引出本文的研究目标，以及这项工作在提升硬件通用性和能效方面的现实意义。

第 2 章是背景和相关工作。这一章会回顾标准注意力机制的计算步骤和复杂度分析，再系统地梳理滑动窗口、扩展窗口、全局以及随机这几种稀疏注意力机制的原理，最后按不同的技术流派去总结现有稀疏注意力加速器的研究进展，与此同时分析它们各自的优缺点。

第 3 章是混合稀疏注意力的数据流特征分析与调度。这一章会深入分析混合稀疏注意力模式中的数据流和数据重用特性，以及详细介绍为了充分利用硬件潜力而开发出来的数据分割技术（包括序列分割和窗口分割）以及数据重排技术，最后会给出后归一化方法的数学推导流程。

第 4 章是基于可重构条与权重求和模块的解耦架构。这一章会先展示 **ReSA** 加速器在软硬件协同设计上的整体框架，接着说明数据调度器是如何把混合稀疏模式转换成硬件能执行的格式，最后将详细阐述由 2D 网格 PE 阵列、可重构条（简称 **RCBar**）和权重求和模块（简称 **WSM**）这三部分构成的硬件核心架构。

第 5 章是硬件设计。这一章主要阐述 **ReSA** 加速器在微观电路上的具体实现，包括 PE 的四阶段流水线设计、基于查找表（**LUT**）的非线性操作近似方法，以及 **RCBar** 和增强型 **WSM** 的内部逻辑，同时亦会阐释其是如何协同工作以支持四种不同的注意力执行模式。

第 6 章是实验评估。这一章会首先介绍基于 **Verilog** 和 **OpenROAD** 流程的实验设置，包括硬件综合参数和基准测试模型的情况，接着通过在 **Longformer** 和 **BigBird** 的工作负载上做周期精确仿真，从延迟、能效以及资源开销几个维度与 **SALO2** 这个基线进行深入的对比分析。

第 7 章是总结与展望。这章会首先总结 ReSA 加速器的主要研究贡献和实验验证结果，接着客观地指出当前设计还存在的局限性，最后对未来在动态稀疏剪枝支持以及多 FPGA 分布式加速等方向上可能开展的研究工作做进一步展望。

第2章 背景和相关工作

Transformer 模型及其衍生架构在自然语言处理、计算机视觉等领域的成功，根植于其核心组件—自注意力机制强大的全局建模能力。然而，自注意力机制固有的二次计算复杂度使其在处理长序列任务时面临严峻的效率瓶颈。为缓解这一问题，稀疏注意力机制应运而生，但现有硬件加速器在处理混合静态稀疏模式时仍存在灵活性不足的局限。本章将系统梳理相关背景知识与研究工作：首先介绍标准注意力机制的计算原理与复杂度分析，进而阐述滑动窗口、全局及随机等典型稀疏注意力模式，最后综述现有稀疏注意力加速器的代表性工作，并分析其各自的优势与不足，为本文后续提出的 ReSA 可重构加速器设计奠定理论基础。

2.1 标准注意力机制与性能分析

自注意力机制，是 Transformer 模型能发挥出那么强性能的核心基石。该机制摒弃了循环神经网络（RNN）固有的序列时序依赖约束，同时突破了卷积神经网络（CNN）局部感受野的局限，能够直接建模序列中任意两个元素间的关联权重，这样就能高效地把全局上下文信息提取出来，再整合在一起。在深入分析之前，首先需要明确：当前深度学习系统所面临的硬件瓶颈，其根源究竟何在？有必要先了解，基于 Transformer^[1]的衍生模型是如何把这个机制给广泛用起来的。

一方面是在自然语言处理（NLP）这个领域里，凭借双向上下文编码机制，BERT^[17]重构了自然语言预训练的主流研究范式，在情感分析、命名实体识别以及问答系统这些任务上的表现都相当出色。而 GPT^[18]系列，该模型以自回归生成作为核心架构，在文本生成、对话系统以及代码补全诸多方向上，也带来了很大进展。另一方面，在计算机视觉（CV）领域里，Vision Transformer（ViT）^[19]会把图像看成是一堆图像块构成的序列，直接把 Transformer 编码器结构直接迁移应用于视觉任务，相关模型在 ImageNet 数据集上的性能表现已然超越 ResNet 等传统卷积神经网络模型，这就为图像分类、目标检测和分割建立了一个统一的框架。到了多模态这块，如 CLIP^[20]和 DALL-E^[21]这类模型，把文本和视觉注意力给结合到一起，旨在开启零样本学习与图像生成领域的全新发展阶段。可以看到，以上衍生模型不

只是在性能上超过了 RNN 和 CNN，还把应用范围扩展到了推荐系统、生物序列分析以及强化学习这些更广的领域里。不过，支撑这些庞大应用的真正核心，其实全都是高强度的自注意力计算。

从数学表达与计算流水的角度来看，标准自注意力机制的详细计算过程可以严谨地分解为以下几个关键步骤：

步骤 1：线性投影

如图 2.1 所示，首先将输入序列 $X \in \mathbb{R}^{n \times d_{\text{model}}}$ （其中 n 为序列长度， d_{model} 为模型维度）通过三组独立的线性变换，分别投影为查询矩阵 Q 、键矩阵 K 和值矩阵 V 。此步骤实现了矩阵维度的初步转换，其完整的矩阵乘法公式表示为：

$$Q = XW^Q, K = XW^K, V = XW^V \quad (2.1)$$

式中， $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d}$ ——投影用到的权重矩阵；

d ——每个矩阵在投影完成之后的维度大小。

每个矩阵做完投影，形状都会变成 $[n, d]$ ，一般来说这个 d 就正好等于 $d_{\text{model}}/\text{num_heads}$ （ num_heads 为注意力头数）算出来的结果。做这一步投影的时候，会涉及到一个矩阵乘法运算，它的复杂度是 $O(n \cdot d_{\text{model}} \cdot d)$ 。

步骤 2：注意力分数计算

第二步，需要去算注意力分数——具体的做法是，拿查询矩阵 Q 和键矩阵 K 的转置做个乘积，通过这个乘积就能推导出注意力分数矩阵 $S \in \mathbb{R}^{n \times n}$ 。为了有效缓解在深度网络训练过程中可能出现的梯度爆炸问题并稳定优化过程，该分数矩阵会被键向量维度的平方根（ $\sqrt{d}$ ）进行缩放。完整的计算公式如下：

$$S = \frac{QK^T}{\sqrt{d}} \quad (2.2)$$

式中， S ——注意力分数矩阵；

Q ——查询矩阵；

K^T ——键矩阵的转置；

d ——键向量维度。

该点积操作由于需要遍历序列中所有元素对，其复杂度达到 $O(n^2 \cdot d)$ ，是整个机制中最为耗时且引发二次方瓶颈的核心环节。

步骤 3：非线性归一化

在此基础上，对分数矩阵的每一行应用 Softmax 函数进行非线性归一化处理，使得每一行的权重之和为 1，从而得到概率分布形式的注意力权重矩阵 $A \in \mathbb{R}^{n \times n}$ 。对于矩阵中的任意一个元素，其 Softmax 计算公式为：

$$A_{ij} = \text{softmax}(S)_{ij} = \frac{\exp(S_{ij})}{\sum_{k=1}^n \exp(S_{ik})} \quad (2.3)$$

式中， $\sum_{k=1}^n \exp(S_{ik})$ —Softmax 分母，用于归一化；

A_{ij} —注意力权重矩阵中第 i 行第 j 列的元素；

S_{ij} —注意力分数矩阵中第 i 行第 j 列的元素。

该步骤包含复杂的指数运算，复杂度为 $O(n^2)$ 。

步骤 4：加权输出

最终，将该权重矩阵与值矩阵 V 相乘，产生加权值向量输出，完成一次完整的自注意力特征提取：

$$\text{Output} = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (2.4)$$

式中，Output—自注意力机制的最终输出矩阵；

V —值矩阵。

该矩阵乘法操作的复杂度同样为 $O(n^2 \cdot d)$ 。

尽管这种标准的全局自注意力机制在处理短序列任务时表现出了卓越的并行计算效率，但其整体的时间和空间复杂度极高。

时间复杂度分析：总时间开销为四步之和： $O(n \cdot d_{\text{model}} \cdot d) + O(n^2 d) + O(n^2) + O(n^2 d)$ 。对于现代大模型而言，由于 n^2 项占据主导，总时间复杂度可简化为 $O(n^2 d)$ 。

空间复杂度分析：需要进行严格的显存占用推导：存储输入矩阵 X 占用 $O(nd_{\text{model}})$ 空间；投影后的 Q 、 K 、 V 矩阵各占用 $O(nd)$ 空间；最为庞大的开销来自于中间生成的分数矩阵 S 和权重矩阵 A ，它们均为 $n \times n$ 的密集矩阵，因此各自占用 $O(n^2)$ 的显存；最终的输出矩阵占用 $O(nd)$ 空间。综上所述，标准自注意力的总空间复杂度可以严谨地表示为：

$$C_{\text{space}} = 2n d_{\text{model}} + 3n d + 2n^2 \quad (2.5)$$

式中， C_{space} —标准自注意力机制总的空间复杂度；

n —序列长度；

d_{model} —模型维度；

d —投影之后的向量维度。

在处理长序列的任务时， n^2 项这一项会随着序列长度呈二次方的趋势往上涨，如此，显存占用的压力往往就比计算本身还要大，成了更严峻的一个瓶颈。

2.2 稀疏注意力机制

从图 2.2 可以看出，通过把注意力机制给稀疏化，确实能在很大程度上缓解长序列带来的计算负担和显存压力^[22]。稀疏注意力的核心思想在于，不去直接计算完整又庞大的密集分数矩阵，而是先限制查询-键对的计算范围，只去关注序列里面相关性比较高的子集。其背后的基本做法是先把原本密集矩阵乘法转换成采样密集-密集矩阵乘法的形式（简称 SDDMM），再去做稀疏-密集矩阵乘法。由于采用的稀疏模式会稍显不同，现有的静态稀疏注意力在计算复杂度上也会体现出明显的差异，具体对比有下面几个方面。

（1）滑动窗口注意力：从图 2.2(a)能够看到，这种模式会用上序列的局部连续性—其会为每一个查询都把关注范围设成一个固定大小的邻近键窗口（设窗口大小为 w ）。这样就形成了一个带状稀疏矩阵，能把复杂度从原先标准的 $O(n^2d)$ 大幅降至 $O(n \cdot w \cdot d)$ ，在捕获局部依赖方面极为高效。

（2）扩展窗口注意力：如图 2.2(b)所示，该模式在滑动窗口的基础上引入了扩展率（设为 r ）。它在不增加计算量（仍为 $O(n \cdot w \cdot d)$ ）的前提下扩大了感受野范围，使模型能够感知更广泛的上下文。

（3）全局注意力：如图 2.2(c)所示，为了弥补基于窗口模式缺乏全局连通性的缺陷，全局注意力引入了少量的“全局 Token”（设数量为 g ）。这些 Token 关注序列中的所有元素，且所有元素也反向关注 Token，从而以极小的额外复杂度 $O(n \cdot g \cdot d)$ 保留了关键的全局信息。

（4）随机注意力：如图 2.2(d)所示，该模式让每个查询关注序列中固定数量（设为 r ）的随机采样键，其复杂度为 $O(n \cdot r \cdot d)$ ，有助于模型捕获长距离依赖关系。

如表 2.1 所示，静态稀疏注意力^[16]与动态稀疏注意力^[8]在硬件映射上存在本质差异。静态模式具有高度的可预测性和规则的空间拓扑，极大地便利了底层定序与复用；而动态模式虽然潜力更大，但给硬件带来了极大的挑战。

表 2.1 动态稀疏与静态稀疏注意力硬件映射特性对比

对比维度	动态稀疏注意力	静态稀疏注意力
稀疏模式生成	运行时根据输入数据动态计算（数据驱动）	预定义的固定规则或掩码（规则驱动）
硬件映射难度	极高。访存地址不规则，难以预取	较低。具有规则的数据流（如对角线、带状）
内存访问开销	极大。存在大量随机访存，Cache 命中率低	较小。可通过空间局部性优化为顺序或块状访存
负载均衡	困难。不同 Token 的有效计算量差异巨大	容易。计算量分布均匀，易于硬件流水线排布
额外硬件开销	需要复杂的索引生成器、预测器或稀疏路由网络	几乎无需额外开销，仅需简单的控制状态机

如图 2.2 所示，混合静态稀疏注意力机制通过组合上述多种单一静态模式（如 Longformer 组合局部与全局，BigBird 组合局部、全局与随机），能够更全面地捕捉序列中的不同依赖关系。然而，当引入窗口分割技术以适应硬件规模时，必须采用“后归一化”方法以保证数学上的正确性。其完整的数学证明过程如下：

设查询 q_i 经过窗口分割后，在片段 T_1 和 T_2 中计算得到的未归一化部分输出分别为 $\text{Output}_i^1 = \sum_{j \in T_1} \exp(S_{ij}) V_j$ 和 $\text{Output}_i^2 = \sum_{j \in T_2} \exp(S_{ij}) V_j$ ，对应的片段权重为 $W_1 = \sum_{j \in T_1} \exp(S_{ij})$ 和 $W_2 = \sum_{j \in T_2} \exp(S_{ij})$ 。

完整的 Softmax 分母应为 $W = W_1 + W_2$ 。最终的归一化输出可展开推导为：

$$\text{Output}_i = \frac{\text{Output}_i^1 + \text{Output}_i^2}{W} \quad (2.6)$$

$$\begin{aligned}
&= \frac{\sum_{j \in T_1} \exp(S_{ij}) V_j + \sum_{j \in T_2} \exp(S_{ij}) V_j}{W_1 + W_2} \\
&= \frac{W_1 * \left(\frac{\text{Output}_i^1}{W_1} \right) + W_2 * \left(\frac{\text{Output}_i^2}{W_2} \right)}{W_1 + W_2} \quad (2.7)
\end{aligned}$$

由此证明，只要在硬件中分别保存每个独立片段的权重 W_1 和部分输出向量 Output_i^1 ，即可在窗口分割模式下通过一次简单的加权求和无损重构最终结果。

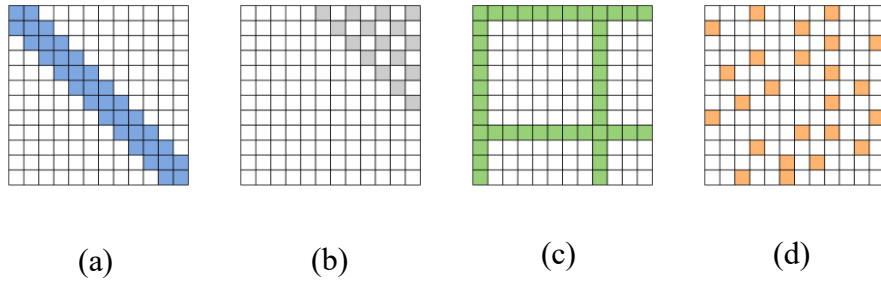


图 2.1 典型的单一静态稀疏注意力模式

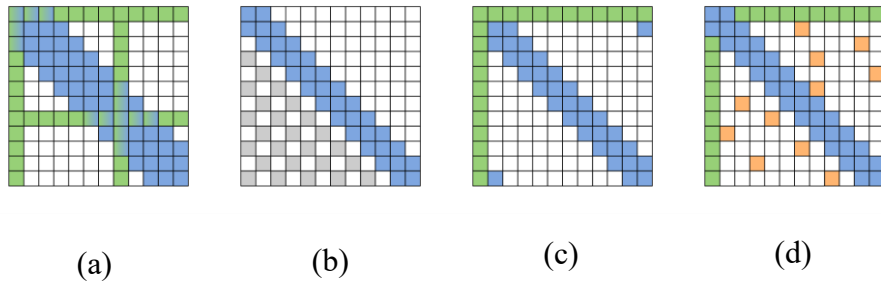


图 2.2 混合静态稀疏注意力机制示例

2.3 相关工作

在 Transformer 的硬件加速这个领域里边，针对稀疏注意力机制的相关研究，已经不再是早期单纯追求省计算量的做法了，而是逐渐演变成了要软硬件协同去优化内存访问的问题——意图把内存访问给消除。为了能把现有技术究竟是如何一步步发展过来的、以及其各自存在哪些局限给剖析得更清楚一些，表 2.2 里会从好几个关键维度出发，对当前一些主流的稀疏注意力加速器做一个比较细致的对比和总结。

在 Transformer 硬件加速这个方向上，早期的稀疏注意力加速器以动态剪枝为核心，但这条路很快就撞上了一个硬瓶颈——内存访问会变得特别不规则。下面将结合表 2.2 里的几个代表性工作，具体分析它们各自的架构特点还有局限性。

第一点，对 SpAtten^[8]模型进行分析与探讨。SpAtten 是早期动态稀疏加速器里一个很有代表性的方案，核心创新是构建了一个级联的 Token 剪枝和注意力头剪枝机制，目的是从输入源头把计算量给减下来。不过这种依赖运行时预测的架构有一个致命的问题：里面的预测器不仅占了不小的芯片面积，而且生成的稀疏模式随

机性特别强。这种随机性一出来，底层存储系统的访问地址就完全没法预测，导致内存访问变得特别乱，负载均衡也出现了严重瓶颈，最后剩余的硬件和访存开销会把剪枝所获增益完全抵消。

第二点是 PADE^[9]。为了解决 SpAtten 里面预测器带来的开销，PADE 提出了一个叫“预测消除”的技术。其没有沿用传统的独立预测模块，而是换了一种思路，用了位串行启用融合机制，在数据流过执行单元的期间精准识别并跳过无效计算块。PADE 确实成功去掉了预测器的硬件冗余，但其底层的数据流还是会被动态稀疏不规则性给限制住，建立不起来高效的数据复用通路，所以当遇见密度比较高的动态模式，执行效率上还是会有一个明显的瓶颈。

第三点是 DOTA^[10]。这个加速器从另一个角度切入，提出一个“弱注意力检测”的机制。该加速器会在硬件层面实时去评估每个注意力块对最终结果造成的边际贡献，接着把贡献特别弱（也就是低于预设阈值的）计算块直接跳过。这种做法能实现全模型路径上的稀疏加速，但它核心要解决两个麻烦：一个是需设计出一个延迟极低的硬件比较器网络来实时算阈值，另一个是得在不去进行太复杂的调度逻辑的前提下，把被不规则剔除之后剩下的任务重新均衡地分配给 PE。

第四点看 DynaX^[11]。在细粒度结构优化这块，DynaX 探索了一种动态 X:M 的细粒度结构剪枝。与粗粒度的整行或整列剪枝不一样，DynaX 允许在更细的粒度上去动态调整稀疏结构。这种设计在一定程度上能把硬件的并行效率提上去，同时也能保住模型的精度，但代价就是引入了一套特别复杂的动态路由网络。而且因为 DynaX 的硬件配置不够灵活，很难适应不同层之间差异很大的稀疏率，所以在端侧部署的时候会面临面积和功耗方面的很大压力。

第五点，随着长序列任务的需求越来越多，研究的重心也开始往静态结构化稀疏转移。FlashAttention^[12]奠定了 I/O 感知优化的理论基础，其核心贡献在于严格证明了：在内存受限的硬件里，减少 HBM（也就是高带宽内存）的访问次数，比单纯减少 MAC 操作次数对提升性能更管用。不过需要说明的是，FlashAttention 本质上还是软件层面的一种分块计算算法（也叫 Tiling），该工作未在硬件微架构层面针对混合稀疏模式定制专门的数据通路。

第六点是 ASADI^[13]。在 FlashAttention 这个理论的基础上，ASADI 进一步挖掘了滑动窗口注意力里自带的“对角线局部性”。该研究采用对角线数据格式(DIA)

组织数据，再加上原位计算的技术，能把滑动窗口模式下的随机访问开销给明显降下来，从而在这个单一模式上做到极致优化。但问题也很明显：这种高度特化的数据流拓扑等于被牢牢焊死在对角线传播路径上，因此，该设计无法有效处理需要全局广播或随机访存的全局注意力与随机注意力模式。

第七点，也是本文最核心参照的基线，即 SALO2^[16]加速器。该加速器初步解决了静态模式的映射问题，其提出的对角线重用数据流在纯局部场景下效率尤其高。不过从表 2.2 的对比能清楚看出，SALO2 对混合模式的支持完全依赖于所谓的“软件层面的完美分区调度”。一旦碰到配置参数不是最优的混合场景（比如 BigBird 里面高密度的随机 Token），由于非局部注意力和局部注意力的数据流之间是强耦合的，SALO2 的处理效率就会严重依赖特定的硬件配置。只要全局或者随机 Token 的空间分布没能与局部注意力的移动模式对齐，这种数据复用机制就会直接崩溃。到了这个时候，SALO2 就不得不单独去处理这些非局部的组件，导致硬件冗余特别严重，资源利用率也跟着急剧降低。所以，如何设计出一种能够原生支持混合模式、同时拥有较高架构灵活性的加速器，依然是目前这个领域里一个急需解决的关键问题。

表 2.2 主流稀疏注意力加速器特性对比

加速器名称	稀疏类型支持	混合模式支持	核心优化策略	硬件开销与灵活性
SpAtten	动态稀疏	否	级联 Token 与头剪枝	预测器引入严重硬件冗余，访存极不规则
PADE	动态稀疏	否	预测消除与阶段融合	消除了预测器冗余，但受限于动态模式效率
DOTA	动态稀疏	否	弱注意力检测与跳过	减少了边际贡献计算，但负载均衡设计复杂
DynaX	动态结构化	否	动态 X:M 细粒度剪枝	提升了并行效率，但硬件配置不够灵活
FlashAttention	静态密集/稀疏	有限	I/O 感知优化理论	软件级优化，未针对混合模式定制硬件数据流
ASADI	静态局部	否	对角线局部性与原位计算	对滑动窗口极致优化，无法处理全局/随机模式
SALO2	静态混合	软件调度支持	对角线重用数据流	高度依赖完美分区调度，非最优配置下资源闲置严重

2.4 本章小结

本章系统梳理了 Transformer 模型核心组件——自注意力机制的基本原理与计算特性，并深入分析了其在长序列任务中因二次复杂度所引发的计算与访存瓶颈。在此基础上，详细介绍了稀疏注意力机制的主流范式，包括滑动窗口、扩展窗口、全局及随机注意力，并对比了静态稀疏与动态稀疏在硬件映射过程中的本质差异。

通过对现有稀疏注意力加速器（SpAtten、PADE、DOTA、DynaX、FlashAttention、ASADI 及 SALO2 等）的综述与分析可知，尽管上述工作在单一稀疏模式或特定场景下取得了显著的能效提升，但在面对融合多种模式的混合静态稀疏注意力（如 Longformer、BigBird）时，现有架构普遍存在数据流僵化、资源利用率不均及硬件冗余严重等固有问题。特别是以 SALO2 为代表的基线方案，由于将非局部计算与局部数据流强耦合，其性能高度依赖软件分区的完美调度，难以在非最优配置下维持高效执行。

因此，如何在硬件层面打破不同稀疏模式之间的计算依赖，实现混合稀疏注意力的高效、灵活映射，已成为当前 Transformer 加速器设计中的关键挑战。针对上述问题，下一章将从数据流特征分析入手，提出面向混合稀疏注意力的数据分割与重排技术，为实现解耦且可重构的硬件架构奠定基础。

第3章 混合稀疏注意力的数据流特征分析与调度

针对第2章末尾指出的 SALO2 架构中非局部注意力与局部注意力“数据流强耦合”导致的资源闲置与硬件冗余问题，本章从软件与硬件交界处的数据流入手，提出 ReSA 加速器的数据调度与重塑机制。该机制的核心在于数据解耦，其工作原理为：在数据输入底层硬件之前，先执行一套灵活的调度策略，从而将不同稀疏模式之间的非规则数据依赖关系彻底剥离，这样就能打破原有架构里的限制：非局部计算必须依附于局部数据流才能正常运行。将这一限制打破后，为后续构建高度灵活的硬件架构奠定了坚实基础，因为数据供给已经变成结构化和规则化的。可以从图 3.1 看到，本章讲的数据重塑策略主要分成两个阶段来进行，一个阶段是数据流特征分析，另一个阶段是数据分割重排。数据流特征分析这个阶段，主要是旨在深入分析不同混合稀疏注意力模式（像滑动窗口、膨胀窗口、全局还有随机这些）内部所蕴含的数据复用规律；而到了数据分割重排这个阶段，就会在前面分析的基础上，用上序列分割、窗口分割还有针对性的数据重排序技术，把非规则的混合稀疏模式给转化成标准化的计算块，让它们能与 PE 阵列的物理规模对上号，这样就能在满足数据流约束和硬件可扩展性的前提下，把整个系统的计算吞吐率给拉到最大。

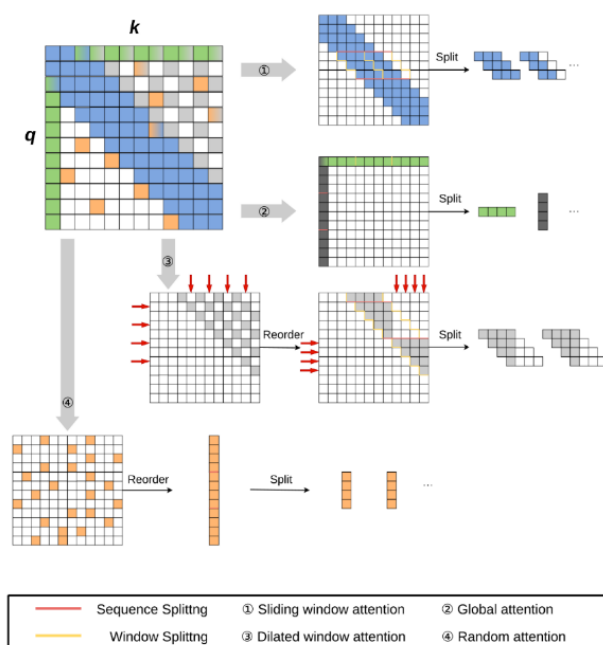


图 3.1 混合静态稀疏注意力模式的数据分割与重排技术

3.1 数据流与数据重用特性分析

数据调度要做的，本质上就是把混合静态稀疏注意力机制从软件层面转换为加速器可执行的格式，同时还得严格满足底层数据流的时空约束，以及硬件规模的可扩展性要求。若直接将 Longformer、BigBird 等模型中原始的非规则稀疏矩阵输入至二维计算阵列中，硬件 PE 之间的数据交互就会变得尤为离散，导致系统依旧没法高效地复用数据，进而造成资源闲置，访存开销显著增加。例如，在处理膨胀窗口或者随机注意力的时候，不加任何干预，查询（Q）和键（K）之间的交互就会把局部性特征给丢掉。所以，需要先把混合稀疏注意力里不同子模式的数据重用特性给剖析清楚，这样才能指导后面的硬件数据通路该如何设计。

在混合稀疏注意力模式里面，不同的子模式展现出来的数据重用特性差别很大。一方面，对于滑动窗口注意力这种机制来说，连续查询（Q）token 之间是有很大的键（K）向量重用潜力的。为了把这一点直观地展示出来，假设当前窗口大小 $w=4$ ，查询向量序列为 $q_1, q_2, q_3, \dots$ ，键向量序列为 $k_1, k_2, k_3, k_4, k_5, \dots$ 。当查询 q_1 在做计算的时候，其对应的计算窗口是 $[k_1, k_2, k_3, k_4]$ ；当计算推进到相邻的 q_2 时，该窗口就会按照步长 1 滑动到 $[k_2, k_3, k_4, k_5]$ 。把这两次计算放到一起对比就会发现，在连续的计算周期里，相邻查询 q_1 与 q_2 之间只差了 1 个 K 向量（也就是 k_1 和 k_5 不一样），剩下的 $w-1=3$ 个 K 向量（也就是 k_2, k_3, k_4 ）是可以被直接拿来复用的。

为了能在硬件的物理阵列里把这种复用规律给高效地捕获到，本设计针对滑动窗口模式专门配置了一个特定的数据流动方向。从图 3.2 可以看出，在二维 PE 阵列里面，Q 向量会被配置成沿着水平方向（也就是从左到右）依次流进相邻的 PE 里面去；而与它匹配的 K 向量（以及后面跟着的 V 向量），则会被配置成沿着对角线方向（也就是往右下方）流进去。这种“Q 走水平传播、K 走对角线传播”的交错式数据流设计，能够确保当 q_2 进入到下一个 PE 的时候，上一个周期被锁存起来的 k_2, k_3, k_4 恰好也能通过对角线网络同步到达，这样就能在无需额外做访存的条件下实现完美的脉动复用，从而把片上数据重复搬运的开销明显降低。

不过，为了能够克服现有基线方案（如 SALO2 这种）在硬件面积开销和混合模式灵活性上面存在的局限，本架构就没有沿用为全局注意力和随机注意力单独

分配专用 PE 的传统做法。所以在这个注意力瓦片内部，局部组件、全局组件还有随机组件都是被设定成独立去计算的。这种设计虽然在逻辑上把不同子注意力模式之间的数据交叉重用给排除掉了，但是因为引入了可重构总线（也就是 RCBaR）这个机制，全局注意力里自带的 Q 向量或 K 向量的广播与复用特性，依然能够得到有效的保留，并且在硬件层面也能得到很好的支持——这样，就能够在不额外增加计算资源的前提下，为混合模式的独立高效调度奠定了数据基础。

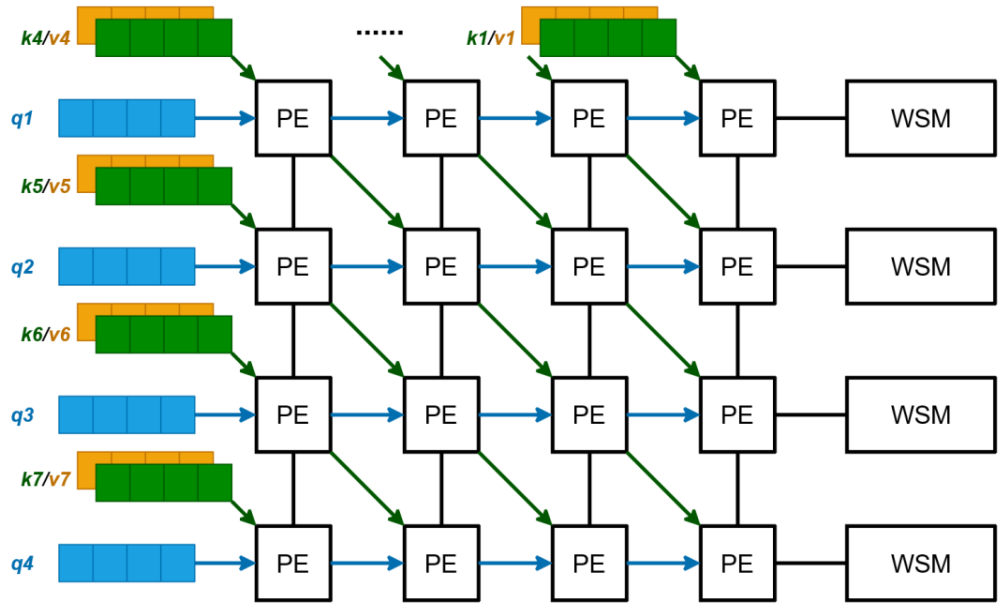


图 3.2 滑动窗口注意力模式在二维 PE 阵列中的数据流示意图

3.2 数据分割与重排技术

为了能在不损失计算精度的前提下，把 Longformer 或者 BigBird 这类模型里面的“非规则稀疏矩阵”给转化成底层硬件阵列能够接受的“结构化数据流”，本研究专门开发了数据分割与重排这一套技术^[16]。在实际应用里，长序列任务的序列长度和滑动窗口尺寸通常都会比 PE 阵列的物理维度要大不少，这就意味着要处理一个完整的窗口或者一段长序列，往往得靠多次计算传递才能完成。为了解决这个问题，数据分割技术会分成两个一起配合的阶段来做，一个是序列分割，另一个是窗口分割——它会把原本很大的完整输入矩阵给拆解成一个个比较小的计算块，

并且让这些块的大小与硬件的物理规模精确地对上，这样就能把硬件资源那边规模上的约束给有效缓解掉。

在具体实施的时候，序列分割这一步是沿着查询（Q）这个维度去对矩阵做划分的。因为不同 Q 行的计算彼此之间是相对独立的，所以这么做不仅不会破坏计算本身的完整性，反而能把并行度给明显地提上去。相比之下，窗口分割的做法是把单行完整的计算任务给切成了多个分段，换言之，单个查询在经过不同窗口分段计算之后，会生成多个部分输出向量。这些不完整的计算结果，实际上代表的是 Q 和特定的 K 在不同片段里交互时产生的中间状态，必须得把它们累加起来，才能推导出最终的结果。

3.2.1 后归一化方法的数学推导

为什么窗口分割后不能直接对各个分段做 Softmax？因为 Softmax 的分母包含该查询对所有相关键的指数求和。如果直接分段归一化，每个分段计算出的权重之和均为 1，合并后将导致最终结果被放大。为解决此问题，本设计采用了后归一化方法，即在分割计算阶段保留未归一化的指数和作为权重。

设查询 q_i 在窗口分割后被划分为 T_1 和 T_2 两个片段。在片段 T_1 中，计算出的 Softmax 分母权重为 $W_1 = \sum_{j \in T_1} \exp(S_{ij})$ ，产生的未归一化部分输出向量为 $\text{Output}_i^1 = \sum_{j \in T_1} \exp(S_{ij}) V_j$ ；同理，在片段 T_2 中得到 W_2 和 Output_i^2 。完整的 Softmax 分母应为 $W = W_1 + W_2$ 。根据 Softmax 的定义，最终的归一化输出可以展开为：

$$\text{Output}_i = \frac{\text{Output}_i^1 + \text{Output}_i^2}{W} = \frac{W_1 * \left(\frac{\text{Output}_i^1}{W_1} \right) + W_2 * \left(\frac{\text{Output}_i^2}{W_2} \right)}{W_1 + W_2} \quad (3.1)$$

由此可见，只要在硬件中分别保存每个独立片段的权重 W_1 和部分输出向量 Output_i^1 ，即可在所有分段计算完成后，通过一次简单的加权求和操作无损且高效地重构出最终的正确结果，彻底避免了分段归一化带来的数学错误。

此外，针对膨胀窗口和随机注意力等缺乏直接分割特性的模式，必须通过数据重排技术进行结构转换。膨胀注意力本身不具备内在的数据重用模式，因此必须被重排为类似滑动窗口的模式以复用现有的高效数据流机制。以膨胀率 $d=2$ 为例，原始矩阵中 q_i 的计算键为 $[k_0, k_2, k_4, k_6]$ ， q_{i+1} 的计算键为 $[k_1, k_3, k_5, k_7]$ ， q_{i+2} 的计算

键为 $[k_2, k_4, k_6, k_8]$ 。这种交织分布无法复用。通过数据重排，具有相同步长间隔的查询组合在一起，形成新的逻辑序列 $[q_0, q_2, q_4]$ 和 $[q_1, q_3, q_5]$ 。在重组后的序列中， q_0 对应 $[k_0, k_2, k_4, k_6]$ ， q_2 对应 $[k_2, k_4, k_6, k_8]$ 。此时，相邻查询（ q_0 与 q_2 ）之间出现了3个重叠的K向量（ k_2, k_4, k_6 ），完美形成了与滑动窗口高度相似的数据重用规律，从而可以直接复用前述的对角线数据流。而对于完全没有内在数据重用的随机注意力，本设计采用专用的硬件执行模式，在软件调度阶段对随机采样的键进行重排，将其物理上离散的交互重组为规则的矩阵结构。通过上述多维度的调度与重塑手段，本章成功将复杂的混合稀疏模式转化为硬件可高效执行的结构化数据流，从而为底层计算阵列的高吞吐量运行提供了坚实的数据保障。

3.3 本章小结

本章围绕混合静态稀疏注意力机制在硬件映射过程中面临的数据流僵化与非规则访存问题，系统地提出了一套面向 ReSA 加速器的数据调度与重塑机制。首先，深入分析了滑动窗口、膨胀窗口、全局及随机等不同稀疏子模式内部的数据重用特性，揭示了局部注意力中存在的对角线数据复用规律，以及非局部注意力因缺乏空间局部性所导致的数据依赖离散化问题。在此基础上，本章设计了序列分割与窗口分割相结合的数据划分策略，并引入后归一化方法，从数学上严格证明了分段计算结果可通过加权求和无损重构为完整的 Softmax 输出，从而为窗口分割下的硬件实现提供了理论支撑。针对膨胀窗口与随机注意力等缺乏固有重用结构的模式，进一步提出了数据重排技术，通过重组查询或键的顺序，将其转化为与滑动窗口高度相似的结构化数据流，进而复用高效的对角线脉动阵列机制。通过上述多维度的调度与重塑手段，本章成功地将复杂的混合稀疏模式转化为规则、可复用且与 PE 阵列物理规模相匹配的结构化计算块，为底层硬件的高吞吐量执行奠定了坚实的数据基础，也为后续第4章所提出的可重构架构设计提供了必要条件。

第 4 章 基于可重构条与权重求和模块的解耦架构

经过第 3 章实施的数据分割与重塑处理，混合稀疏模式在逻辑这个层面上就已经被转化成了规整的、可以被复用的结构化数据块。不过，逻辑层面的这类“数据解耦”要想真正实现、真正把效能发挥出来，还需靠物理硬件层面的“架构解耦”来提供支撑。针对现有稀疏注意力加速器里传统 PE 间互连太僵化、很难适应多种模式数据通路的问题，本章会在前面数据调度机制的基础之上，把 ReSA 加速器在软硬件协同设计上的整体框架给完整地讲清楚。从图 4.1 可以看出，ReSA 框架在宏观架构上突破了传统二维网格阵列只靠水平或对角线上相邻 PE 做数据传递的局限——其通过在硬件底层引入一个专门的、可重构的旁路机制（也就是可重构总线），来彻底承接并实现不同稀疏模式的独立高效计算。这个框架通过让数据调度器与底下的二维 PE 阵列、RCBar 还有增强型 WSM 一起协同工作，能够在不增加额外专用计算单元的前提下，成功地把全局注意力、随机注意力此类非局部的部分从局部注意力刚性的数据流约束里给剥离出来，从而确保混合稀疏任务能在整个硬件流水线里保持高利用率的状态。

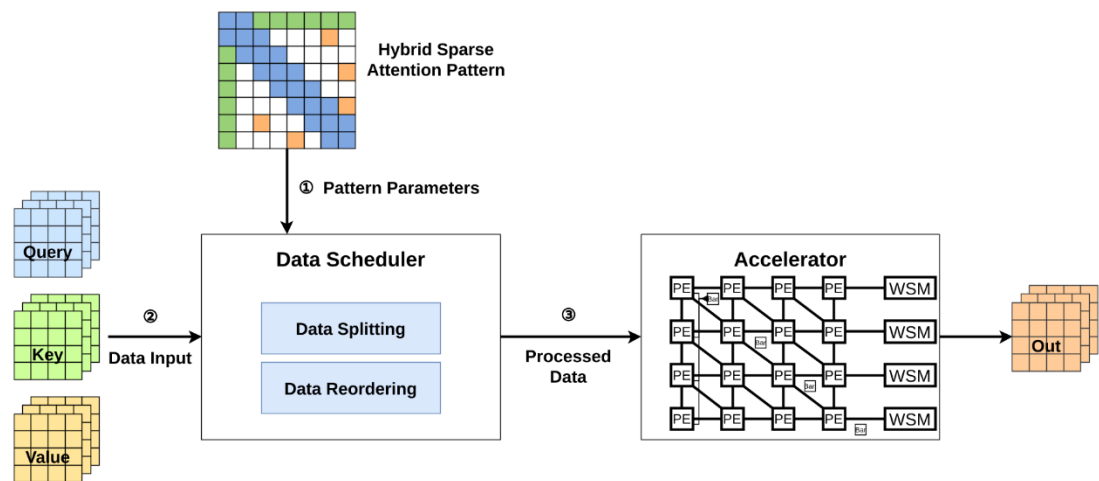


图 4.1 ReSA 加速器的软硬件协同设计整体框架

4.1 整体框架概述

本章把 ReSA 加速器在软硬件协同设计上的整体框架给详细进行了阐述，此框架通过深度的软硬协同机制，成功地把混合静态稀疏注意力模式给高效地加速。一方面来看软件端的数据调度阶段（即第 3 章讲的内容），系统会先根据输入的模式参数，对非规则注意力矩阵进行预处理，用上序列分割、窗口分割还有数据重排这些技术，把它转化成规整的结构化模式，这样就能把现有高效的数据流路径给无缝地用起来。在软硬件交互的那个接口协议层面，软件调度器会把预处理之后的结构化数据块给打包成硬件能够识别的特定指令集，这些指令里会带上当前计算块的稀疏模式标识（比方局部模式、水平全局模式、垂直全局模式还是随机模式），还有向量的物理分段信息以及数据重用的配置参数。硬件那端的顶层控制器在收到这些指令之后，会动态地去解析它们，然后生成细粒度的微操作控制信号，直接去驱动 PE 阵列那边的数据通路选择器、RCBar 的旁路开关，还有 WSM 的计算模式配置，这样就能实现零延迟的模式切换。

另一方面来看硬件加速器这一层，为了能精准地匹配重塑之后多元化的数据流，核心架构采用了高度模块化的设计思路，主要由二维网格 PE 阵列、RCBar 还有增强型 WSM 这三个大部分构成。当输进去的查询、键和值进入到计算阵列之后，局部模式（像滑动窗口和膨胀窗口）会按照那条高效的对角线复用数据流去做流水线计算，从而把片上数据的复用率给最大化。与此同时，全局模式和随机模式不同，该模式会改变传统的数据传递路径，转而利用 RCBar 去实现跨处理单元的灵活数据分配，以及按列来做的并行结果收集，这样就能有效地绕过传统固定互连拓扑带来的通信瓶颈和约束。最后，各个模式算出来的中间结果会有序地被送到权重求和模块里去，这个模块负责做后处理归一化和加权求和，从而把最终的输出无损地重构出来。总的来说，这个架构通过把对专用非局部计算单元的需求给消除掉，在大幅削减整体硬件面积开销的同时，也确保了对各种各样复杂的混合稀疏模式都能提供强大的原生支持，并且保持很高的资源利用率。

4.2 核心硬件架构

ReSA 加速器在硬件层面的核心，是一个可扩展的二维网格 PE 阵列；为了能把局部注意力和非局部注意力之间差异化的数据通路给兼顾，该二维计算阵列在物理结构上进行了较为精细的定制化设计—具体而言就是，每一列都会配上一个可重构条（RCBar），用来实现通用性比较高的数据分布，而每一行则会配上一个增强型的权重求和模块（WSM）。在 PE 阵列的内部，集成了水平和对角线两种互连网络，主要是为了促进滑动窗口这类局部模式下面的数据复用。与此同时，RCBar 作为一个关键的旁路通信枢纽，会直接连到其所在那一列里的每一个 PE 上，这样就把传统固定互连的拓扑限制给彻底打破，从而能够确保对各种高度多样化的数据分布模式都提供支持。

在这个设计里，为了能在统一的硬件物理层面上去高效地处理复杂的混合任务，稀疏注意力计算被严格地分成了四种不一样的执行模式，每一种模式都会走完完全不同的数据分布路径和数据收集路径。下面就是针对这四种模式，分析它们各自完整的数据流转生命周期情况。

4.2.1 局部注意力模式的生命周期

从图 4.2 可以看出，在局部注意力这种模式下，数据流会严格地按照脉动阵列高效复用的原则来走。查询矩阵里的元素（也就是 Q ）会从 PE 阵列最左边的端口 Q_{in} 输入，然后沿着水平方向依次往右边传播；键和价值矩阵里的元素（ K 和 V ）则会从阵列的左上角进去，流向对角线方向的相邻 PE。图中用不同颜色的箭头清晰标示了数据流动路径：蓝色箭头代表的是 Q 在做水平传播，绿色箭头代表的是 K 和 V 在对角线方向上传播。

在每个时钟周期里，进入到 PE_{ij} 的 Q 元素和 K 元素会先做一次乘加运算（即 MAC），算出的注意力分数在经过后续流水线阶段的指数和归一化处理之后，会与同步流进来的 V 元素做一次加权乘法。从图的右侧中能够看到，加权之后的部分和会在行内遵循权重驻留这个原则，通过水平互连网络往右边累加着传递过去。当数据流抵达最右侧的 PE 时，完整的行输出结果就会被直接送到这一行末端的权

重求和模块（即 WSM）里面去，用来进行由窗口分割带来的后归一化加权合并操作。

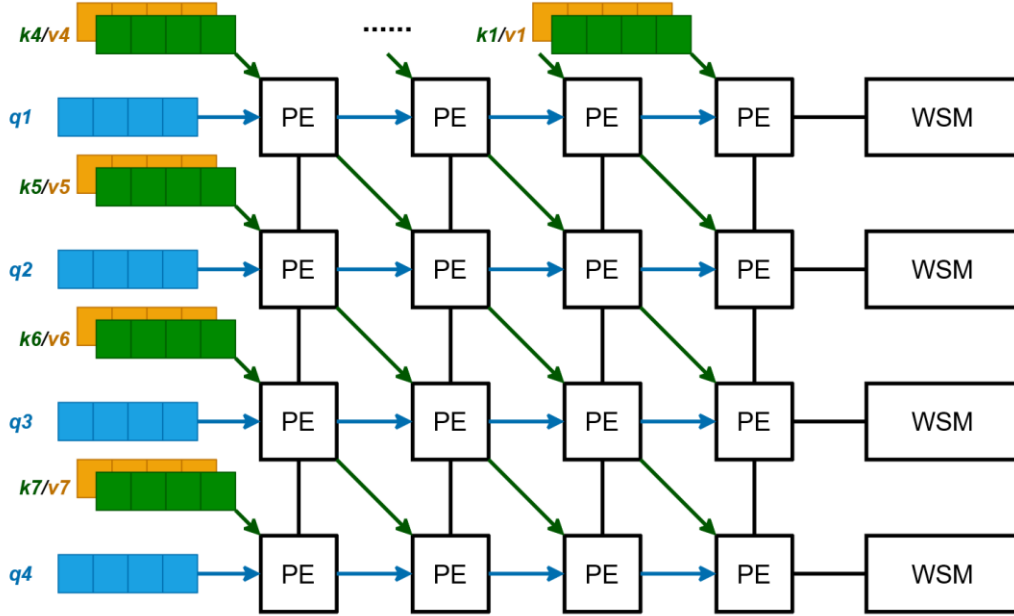


图 4.2 局部注意力模式数据流示意图

4.2.2 水平全局注意力模式的完整生命周期

如图 4.3 所示，水平全局注意力模式下，系统通过数据展开策略利用同一列内的 PE 实现并行计算。完整的 Q 向量首先被分割为多个细粒度段，随后通过常规的水平互连网络分发到同一行的不同 PE 中；而需要与该 Q 向量进行交互的完整 K 和 V 向量段，则绕过常规的水平或对角线通路，直接由该列配备的 RCBar 进行并行分发：RCBar 将 K/V 段精准注入到对应行的 PE 中，与水平流入的 Q 段执行锁存的点积运算。

在后续的 Softmax 与加权阶段，由于同一个 Q 向量所对应的所有 K/V 计算均发生在同一行内的不同 PE 上，原有的水平累加数据流得以完整保留。如图中底部所示，各行产生的部分输出最终被统一路由至一个集中的 WSM 中进行汇聚，并执行最终的归一化处理，从而完成该模式下的全部计算流程。

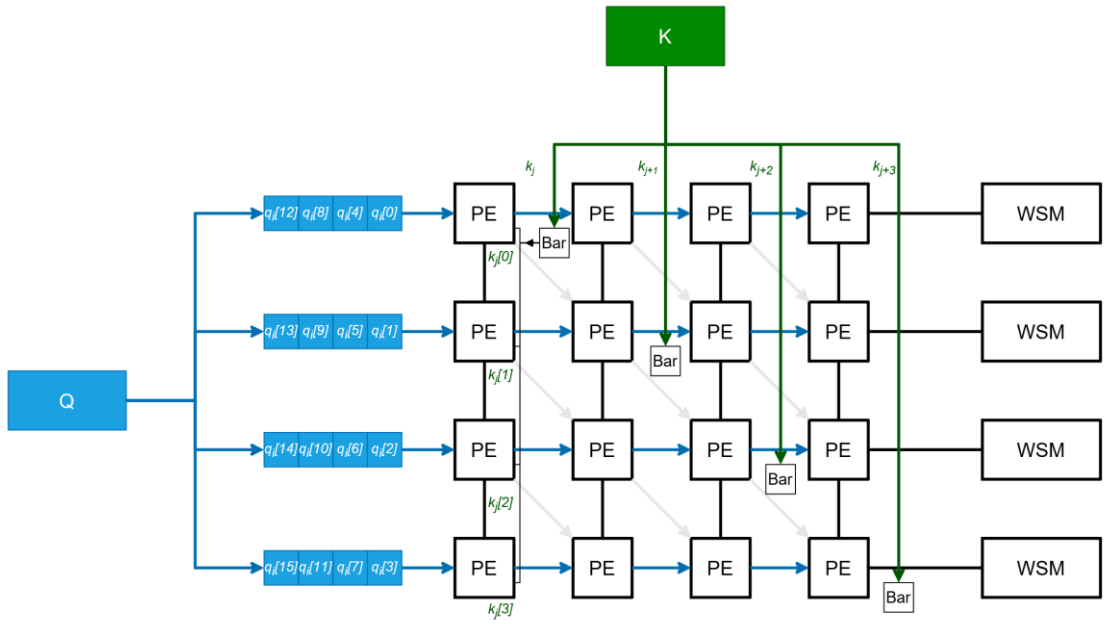


图 4.3 水平全局注意力的数据分布示意图

4.2.3 垂直全局注意力模式的完整生命周期

从图 4.4 能够看到，垂直全局注意力这种模式，在数据通路上与水平全局模式之间是一种镜像对称的关系。完整的 K 和 V 向量段会通过常规的水平互连网络，流到同一行的各个 PE 里面去，然后沿着水平方向做传播；而与这些 K 、 V 向量做交互的 Q 向量段，则是由 RCBar 沿着垂直方向分发到这一列里的每一个 PE 上。图中用绿色箭头和垂直分发路径把这种镜像式的数据流向给清楚地展示出来。

在计算的过程中，因为完整的注意力分数被物理上拆开，并且存储在了同一列不同 PE 的寄存器里，所以系统必须在做指数运算之前，先通过 RCBar 去执行一次垂直累加，这样才能把完整的注意力分数给重新组合起来。从图的中间阶段能够看到，RCBar 会把这一列里各个 PE 的部分累加结果给汇集起来，等分数重组完成之后再分发回给各个 PE，接着去做后续的指数运算。

到了最后的加权求和阶段，每个 PE 算出来的 V 向量加权乘积就不再走传统的水平通路了，而是向上传递给 RCBar。RCBar 作为一个高速的旁路通道，会把分散在各处的这些乘积结果收集起来，然后直接路由到这一行末尾的 WSM 那里，去进行一个简单的元素级累加更新——这样，垂直全局注意力模式的全部计算流程就完成了。

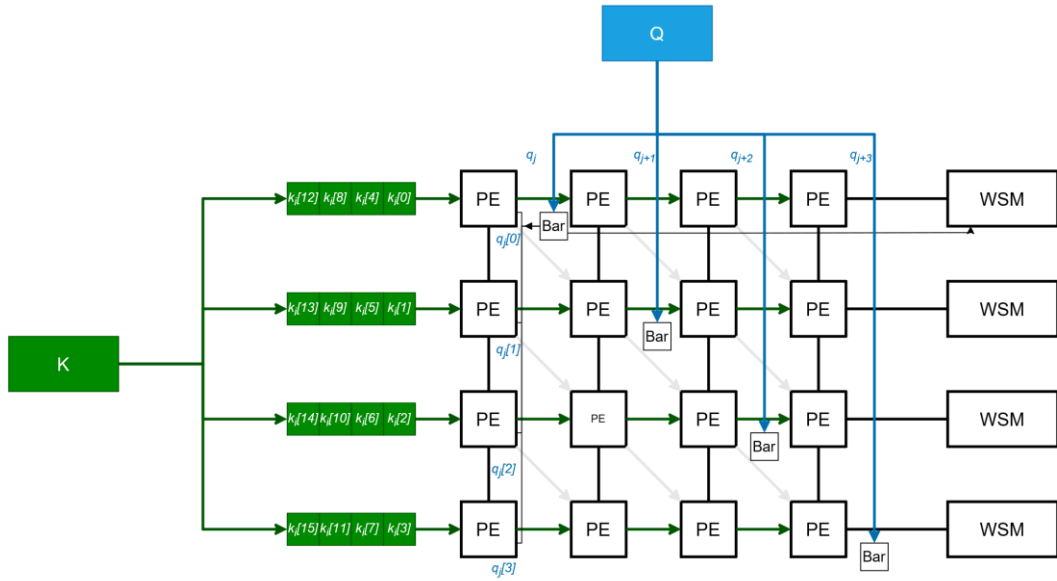


图 4.4 垂直全局注意力的数据分布示意图

4.2.4 随机注意力模式的完整生命周期

如图 4.5 所示，由于随机注意力缺乏任何空间局部性，其数据流转完全依赖 RCBar 的灵活交织分发能力。在点积计算这个阶段，Q 和 K 这两个向量段都不再通过水平或对角线上相邻的 PE 去做传递，而是由 RCBar 按照随机索引的顺序，以一种交替且离散的方式注入到同一列的各个 PE 里，实现一种原地锁定的计算效果。图中用玫红色的离散箭头和随机化注入路径，把这种非规则的分发机制给直观地展示出来。在指数运算开始之前，同样会通过 RCBar 去执行一次列内的垂直累加，以此来重组出完整的注意力分数。到了加权求和阶段，V 元素与指数分数相乘之后的结果，会再次通过 RCBar 直接旁路到 WSM 那边去。从图的右侧可以看出，因为随机模式算出来的输出在数学上与其他模式是相互独立的，所以 WSM 在收到这些数据之后，只需要做没有权重的元素级累加就行。

此外，为了能在交织执行的混合场景里把硬件利用率给最大化，同时也能避免流水线里边出现气泡，系统会在控制器这个层面强制执行一个比较严格的计算顺序。具体来说就是，针对每一个经过序列分割之后得到的计算段，垂直全局注意力和随机注意力会被赋予最高的处理优先级。这么设定优先级的根本原因在于：垂直全局模式和随机模式算出来的输出，在 WSM 里面只需要做一种很简单的元素级

累加（完全不需要归一化加权），而且因为它们非局部的 **Token** 密度本身就很低，所以单次计算下来的吞吐量也会比较小。把这样的计算安排在高密度的局部计算后面去做，**WSM** 里面可能已经堆积着一些正在等归一化的局部输出——这个时候如果硬要把非局部的结果写进去，就会引发数据依赖方面的冲突，否则就得去引入一套很复杂的权重隔离逻辑。所以，让低密度的非局部计算率先完成，并且通过旁路把结果送到 **WSM** 去做简单的累加，这样就能为后面高密度的局部注意力计算腾出完整的阵列资源和 **WSM** 资源，从而把模式切换时的流水线停顿给彻底消除掉，最终在复杂的混合稀疏配置下也能始终维持一个极高的系统吞吐量。

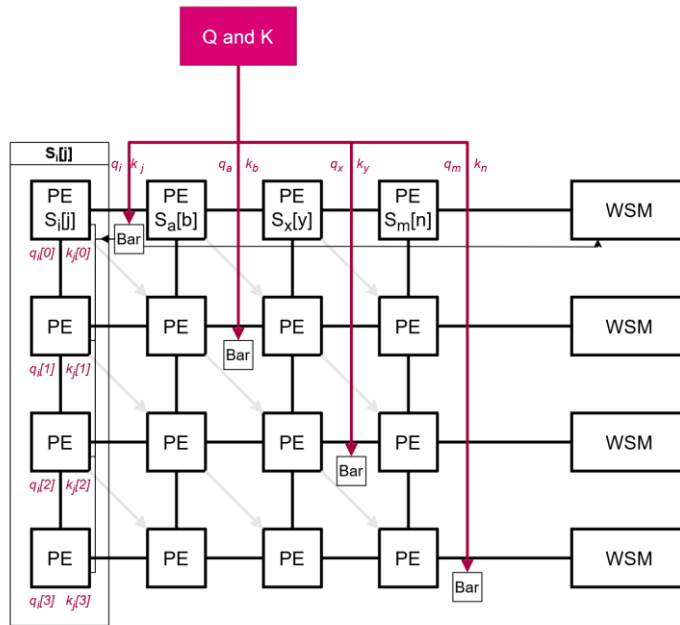


图 4.5 随机注意力的数据分布示意图

4.3 本章小结

本章在第 3 章数据调度与重塑机制的基础上，进一步从物理架构层面阐述了 **ReSA** 加速器的可重构硬件设计。针对现有稀疏注意力加速器中传统 **PE** 阵列互连拓扑僵化、难以适配多种数据通路的问题，本章提出了基于可重构条（**RCBar**）与增强型权重求和模块（**WSM**）的解耦架构。该架构将混合稀疏计算严格划分为局部、水平全局、垂直全局及随机四种独立的执行模式，每种模式均配备差异化的数

据分布与收集路径。具体而言，局部注意力模式沿用高效的对角线脉动数据流，实现 Q 向量的水平传播与 K/V 向量的对角线传播；水平全局注意力模式通过数据展开策略，利用同一列内的 PE 实现并行计算，并由 RCB_{ar} 完成 K/V 向量的列内分发；垂直全局注意力模式则呈镜像对称，由 RCB_{ar} 负责 Q 向量的垂直分发与列内累加重组；随机注意力模式完全依赖 RCB_{ar} 的灵活交织能力，以原地锁定方式完成非规则计算。在此基础上，本章还设计了 PE 内部四阶段流水线（点积计算、LUT 近似指数运算、对齐累加、加权求和），并通过 RCB_{ar} 内部的广播器、排序器与汇聚器等纯组合逻辑模块，实现了低开销的跨 PE 数据旁路与模式切换。增强型 WSM 则通过参数化配置，能够在后归一化加权累加与元素级直接累加之间无缝切换，从而在系统级层面消除了对专用非局部计算单元的需求。上述微架构层面的可重构设计，成功地在物理电路上实现了不同稀疏模式之间的计算解耦，为混合静态稀疏注意力机制的高效、灵活硬件加速提供了完整的解决方案。

第 5 章 硬件设计

经过第 3 章的数据分割与重排，混合稀疏模式在逻辑层面上已被转化为规整的、可复用的计算块。然而，在物理电路层面，传统的 PE 阵列受限于僵化的互连拓扑，其内部及相邻 PE 之间通常仅支持单向的水平或对角线数据传递。这种固化的物理数据通路显然无法满足第 3 章提出的“垂直全局”和“随机”等非局部模式所要求的跨行、跨 PE 灵活数据分发与按列结果收集需求。为了使逻辑层面的“数据解耦”真正实现为物理电路层面的“架构解耦”，本章将深入微观层面，详细阐述 ReSA 如何通过重构 PE 的内部四阶段流水线、引入可重构条以及增强型 WSM，在几乎不增加额外计算资源的前提下，实现对局部、水平全局、垂直全局及随机这四种执行模式的原生电路级支持。

5.1 稀疏注意力加速器架构与数据布局

本章把 ReSA 这个稀疏注意力加速器在具体硬件设计上的设计方法给详细阐述。与现有的基线架构（比如 SALO2）不同——SALO2 主要是针对某几种预先定义好的静态稀疏模式去做优化——本文提出的这个加速器架构，本质上旨在一个更广的范围里，对各种静态稀疏注意力模式都能实现比较高的执行效率。具体而言，这个架构能够独立且高效地去处理包括滑动窗口、全局注意力、随机注意力还有膨胀（即扩展）注意力在内的各种混合模式，这样底层硬件在面对不同模型架构的时候，灵活性就能得到明显增强。

在物理拓扑这个层面上，此加速器的核心是一个可扩展的二维网格 PE 阵列。本设计把 PE 阵列的规模确定为 8×8 ，这个规模基于严谨的硬件资源与计算延时的 Trade-off 分析而确定：一方面， 8×8 这个阵列规模已经足够容纳典型长序列任务里单个注意力头的维度（例如 64 或者 128），能够确保单次的数据分发就把整个阵列填满，从而把并行度最大化；另一方面，在这个规模下面，RCBar 和 WSM 的扇出负载也能被控制在一个合理的区间里，不会因为关键路径太长而导致无法满足 1 GHz 时钟频率这个约束。为了能协同匹配多样化的数据通路，每一列都会配上一个可重构条，用来实现通用性比较高的数据分配；每一行则会配上一个权重求和模

块，用来做后处理的归一化操作。PE 阵列的内部集成了水平和对角线两种互连网络，主要是为了促进局部模式下面的数据复用；与此同时，每一个 RCBBar 作为一个关键的旁路通信枢纽，会直接连到其所在那一列里的每一个 PE 上，这样就把传统固定互连的拓扑限制给彻底打破，从而能够确保对各种高度多样化的数据分布模式都提供支持。

5.1.1 RCBBar 内部结构设计

为了实现上述跨 PE 的灵活数据分发与收集，本研究设计了专用的 RCBBar 硬件结构如图 5.1 所示。从微观电路逻辑来看，RCBBar 内部不包含任何乘加计算单元，仅由三部分关键组合逻辑构成：

（1） 数据选择器与位宽移位器

负责接收来自外部存储或相邻 PE 的数据，并根据当前配置的注意力模式，动态选择需要旁路传输的数据段，并执行必要的位宽截断或对齐移位操作。

（2） 广播器

在 Stage2 中，当列内 PE 完成部分点积累加后，广播器负责将完整的累加结果同步分发至该列的所有 PE 中，为后续的指数运算提供统一输入。

（3） 排序与汇聚器

在 Stage3 和 Stage4 这个阶段，针对垂直全局模式以及随机模式，排序器会把同一列里不同 PE 算出来的稀疏指数值（亦或是加权之后的结果）按照原始序列的顺序重新排序，通过汇聚器把它们打包成连续的数据流，再高速地抛射到这一行末尾的 WSM 里面。这种纯逻辑、完全不涉及计算的设计，本质上确保了 RCBBar 在面积和功耗上的开销都非常小，但却能从根本上去解耦不同注意力模式之间空间上的数据依赖关系。

在这个架构里面，为了能在统一的物理阵列上把复杂的混合任务给高效地处理掉，稀疏注意力计算被严格地划分成了四种截然不同的执行模式，每一种模式都会走完全不同的数据分布路径和数据收集路径。第一点是针对局部注意力模式（这里面包括滑动窗口模式和膨胀模式），系统会沿用高效的脉动数据流——也就是让查询矩阵（Q）沿着水平方向去做分布，而键矩阵（K）和值矩阵（V）则是沿着对角线的方向流进空间阵列里。第二点是针对水平全局注意力、垂直全局注意力还有随

机注意力这些模式，因为缺乏能够被相邻 PE 直接复用的数据特征，所以设计上创新性地引入了一个数据展开的策略：把完整的输入向量给拆成多个细粒度的段，然后分配给同一列里的每一个 PE，这样就能实现细粒度的并行计算。第三点是，为了能在交织执行的混合场景里把硬件利用率给最大化，同时也能避免流水线里边出现气泡，系统会在控制逻辑这个层面强制执行一个比较严格的计算顺序。具体来说，针对每一个经过序列分割之后得到的计算段，垂直全局注意力和随机注意力会被赋予更高的处理优先级—这种调度策略本质上就是为了确保低密度的非局部计算能够率先完成并且通过旁路输出出去，从而为后面高密度的局部计算腾出阵列资源，进而在复杂的混合稀疏配置下也能始终维持一个极高的系统吞吐量。

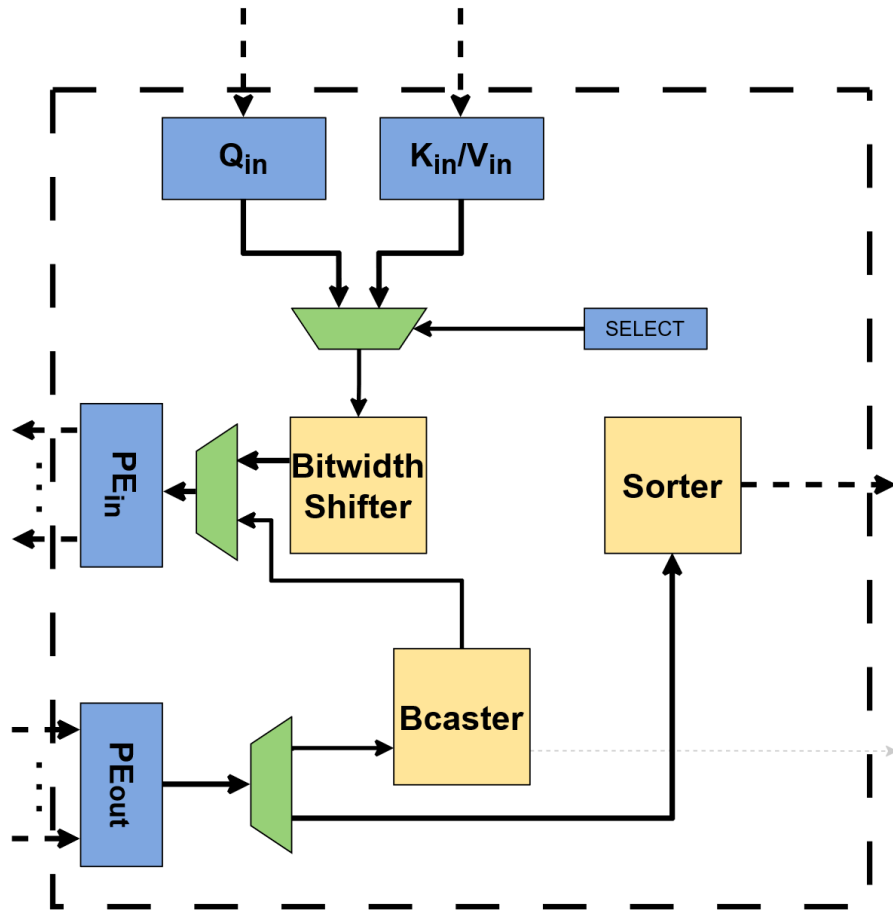


图 5.1 RCBar 内部结构设计

5.2 PE 设计

从图 5.2 可以看出，PE 作为整个加速器最基础的计算单元，其内部集成了一个乘加器（MAC）作为核心的算术引擎，并且采用了四阶段的流水线结构，旨在支持高吞吐量的连续计算。本设计在继承了现有架构高效四阶段流水线的基础之上，又创新性地把第 3 章讲到的数据展开策略和按列累加策略给结合在一起。图中把四个流水阶段各自的数据路径和控制逻辑都标得很清楚：Stage1 负责的是 Q 和 K 之间的点积计算以及动态路由，Stage2 去执行非线性的指数运算并用 LUT 来做近似，Stage3 完成对齐累加再加上 Softmax 分母的计算，Stage4 则进行加权求和然后输出。通过这么一一系列精细化的流水线设计，本架构在把资源开销给降到最低的同时，也实现了很高的硬件利用率以及很不错的多模式兼容性。

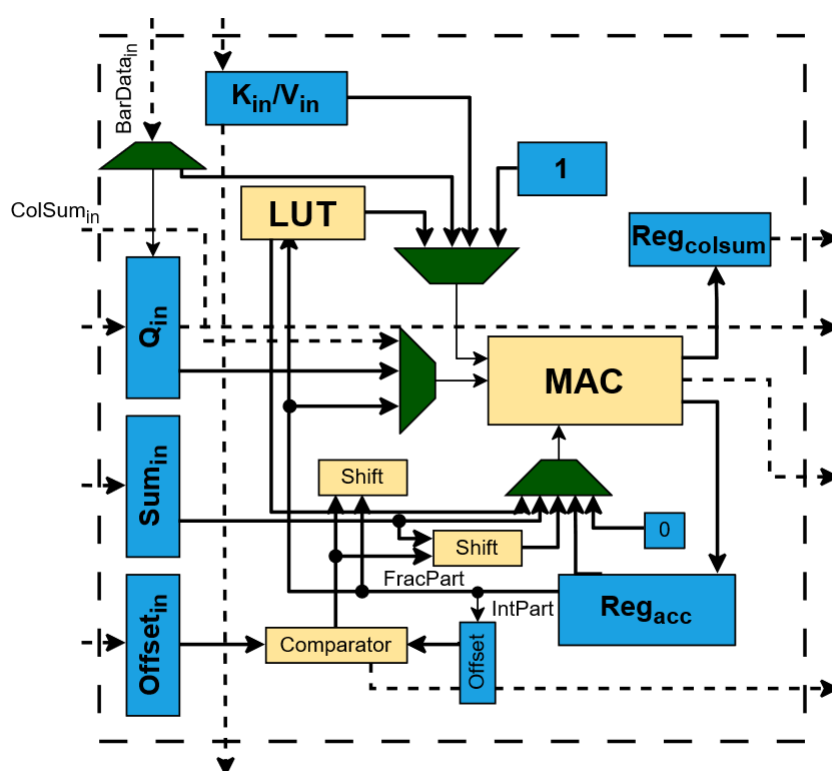


图 5.2 PE 内部结构图

5.2.1 Stage1：点积计算与动态路由

在流水线的第一阶段，查询（Q）和键（K）向量的计算严格遵循输出驻留原则。在每个时钟周期内，PE 接收来自不同通路的数据元素进行相乘，并将结果持续累加至本地寄存器（Regacc）中。当前次向量点积计算完成后，根据系统当前激活的注意力模式，数据元素会被动态路由并传播到不同的目的地。其具体的数据传播逻辑真值表如下所示：

（1） 局部注意力模式

Q 元素传递至右侧相邻 PE，K 元素传递至对角线相邻 PE（复用局部滑动窗口特征）。

（2） 水平全局注意力模式

Q 元素传递至右侧相邻 PE，而 K 元素驻留在当前 PE 中不向外传播（实现同行 K 向量的共享）。

（3） 垂直全局注意力模式

K 元素传递至右侧相邻 PE，而 Q 元素驻留在当前 PE 中（实现同行 K 向量的流动与 Q 的列内锁定）。

（4） 随机注意力模式

Q 和 K 元素均不向相邻 PE 传播，实现数据的原地锁定，完全依赖 RCBar 进行独立分发。

5.2.2 Stage2：非线性指数运算与 LUT 近似

如图 5.3 所示，对于采用数据展开策略的模式（水平全局、垂直全局和随机注意力），由于一个完整的注意力分数被物理拆分并分布在单列 PE 的多个寄存器中，系统必须在计算指数函数之前执行垂直的列累加操作以重组完整分数。图中以垂直累加路径和 RCBar 汇集方式清晰展示了这一重组过程：同一列内各 PE 的部分累积结果通过 RCBar 汇集，完成完整分数的重构。

为了硬件高效实现复杂的非线性操作，本设计采用基于查找表（LUT）的分段线性函数来近似计算指数值^[13]。具体而言，对于重组后的完整注意力分数 S_{ij} ，将其拆分为整数部分和小数部分，即 $S_{ij} = \text{Int} + \text{Frac}$ 。指数运算可转化为：

$$\exp(S_{ij}) = \exp(\text{Int} + \text{Frac}) = \exp(\text{Int}) * \exp(\text{Frac}) \approx 2^{\text{Int}} * (a * \text{Frac} + b) \quad (5.1)$$

式中，Int— S_{ij} 的整数部分；

Frac— S_{ij} 的小数部分；

a—分段线性逼近的斜率；

b—分段线性逼近的截距。

其中，整数部分 $\exp(\text{Int})$ 被近似为 2^{Int} ，在硬件中仅需通过极低开销的移位操作即可实现；而小数部分 $\exp(\text{Frac})$ 则通过 LUT 查表实现。如图 5.3 右侧的 LUT 结构所示，存储在 LUT 中的系数 a（斜率）和 b（截距）根据 Frac 的区间动态获取，随后通过一个乘法器和一个加法器完成线性逼近。这种设计彻底摒弃了昂贵的泰勒展开或 CORDIC 算法，以极小的精度损失换取了数量级的面积与功耗节省。

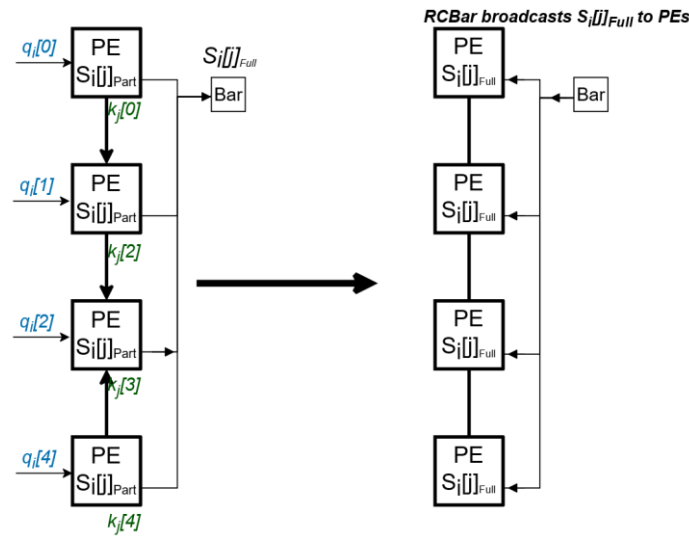


图 5.3 列内部分点积累加过程

5.2.3 Stage3：对齐累加与 Softmax 分母计算

在这个阶段，操作逻辑会根据注意力模式的不同而发生比较明显的分化。

从图 5.4 可以看出，对于局部注意力模式以及水平全局注意力模式来说，同一行里的 PE 会通过水平数据流去依次累加各自的指数值，这样就能拿到 Softmax 的分母（也就是除数）。因为不同 PE 算出来的指数值在量纲上可能会有比较大的差异（本质上是整数部分不一样），所以本设计沿用了 SALO2 里核心的对齐累加逻辑。

辑：每个 PE 在向右传递部分和的时候，会同时传一个 Offset（也就是偏移量，其实就是整数部分）。当前 PE 在收到左侧传过来的部分和之后，会通过比较器把本地的 Offset 和传进来的 Offset 比，然后把较小的那个通过移位器做右移对齐，接着再去做加法操作，最后更新最终的 Offset 再继续向右传递。

而对于垂直全局注意力和随机注意力两种模式，从图 5.5 和图 5.6 可以看出，因为其在单行里需要计算的指数项数量确实太稀疏，所以如果依旧使用水平数据流去做累加，效率上会有比较大的损失。所以，两种模式会换一种方式——通过 **RCBar** 把同一列里各个 **PE** 的公共偏移量和部分和给收集起来，然后直接高速传输到 **WSM** 那边去做最终的累加和更新。图中把这种垂直收集和旁路传输的路径给标得很清楚。

5.2.4 Stage4: 加权求与输出

从图 5.4、图 5.5 还有图 5.6 可以看出，**Stage4** 主要负责处理值（**V**）向量的加权乘法运算。对于局部模式和水平全局模式（即图 5.4 画的情况），其会遵循权重驻留这个原则，在行的内部做水平方向上的流动和累加，最后把结果输出到这一行末尾的 **WSM** 那里去。而对于垂直全局模式和随机模式（图 5.5 和图 5.6 两种情况），其会再次调用 **RCBar** 来充当高速旁路的角色，把 **V** 元素与指数分数相乘之后的结果直接路由到 **WSM** 那边，然后做一种比较简单的元素级累加。

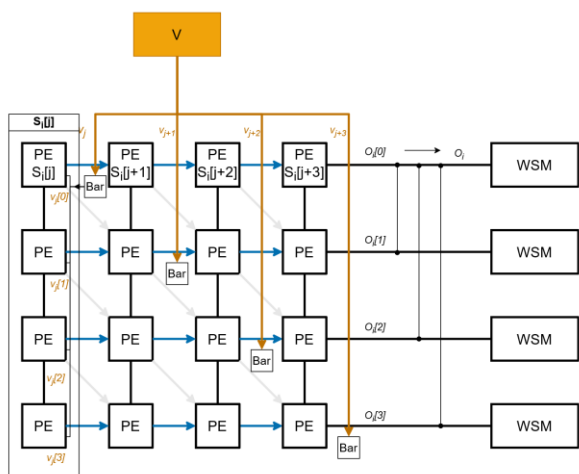


图 5.4 水平全局注意力模式在 Stage 3 与 Stage 4 的数据流示意图

这种精细化的微观流水线设计，本质上使得全局注意力和随机注意力的那部分计算能够被高效又紧凑地映射到单列 PE 上面去，从而把传统架构里为了支持非局部注意力而不得不额外增设专用 PE 组所带来的面积冗余给彻底消除掉。图中用不同颜色的数据流向箭头把各类模式在 Stage4 阶段走的差异化路径给区分开，也比较直观地展示了本架构在解耦设计上的核心理念。

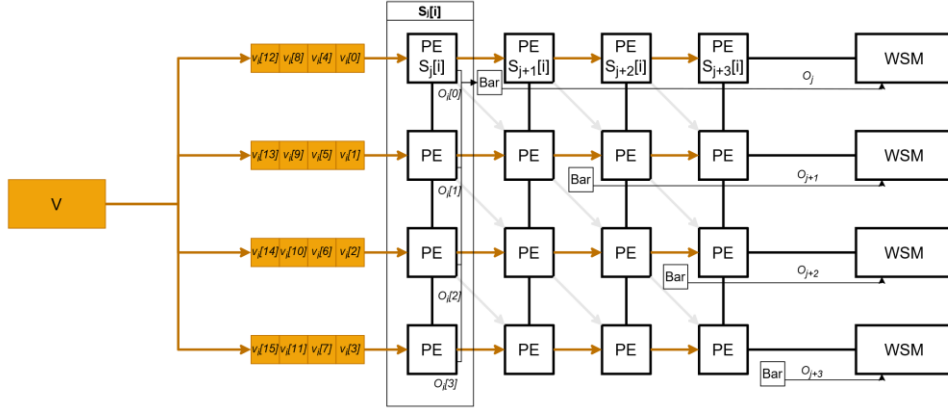


图 5.5 垂直全局注意力模式在 Stage 3 与 Stage 4 的数据流示意图

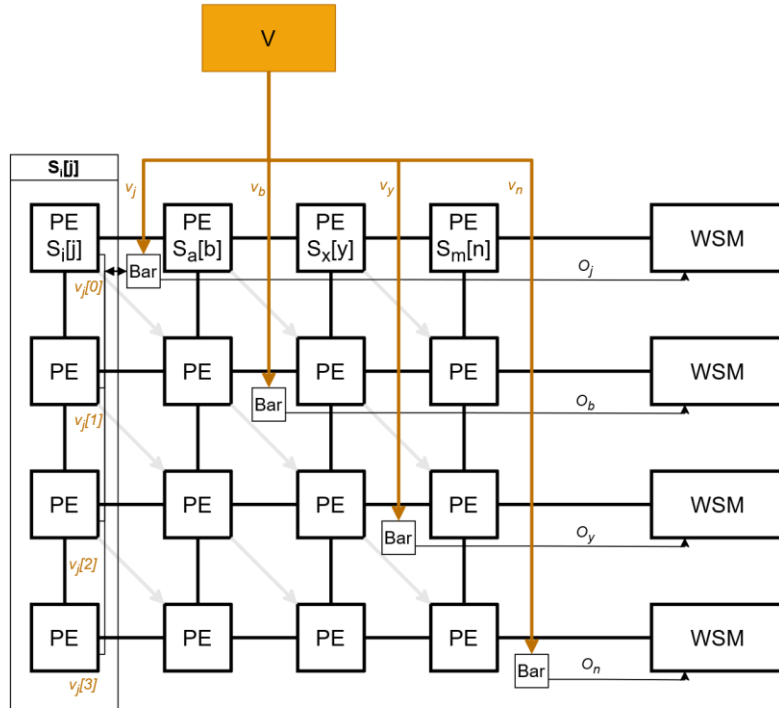


图 5.6 随机注意力模式在 Stage 3 与 Stage 4 的数据流示意图

5.3 增强型 WSM 设计

为了能够适配四种差异显著的执行模式，本研究对传统的 WSM 做了深度的增强。所设计的 WSM（加权求和模块）不仅需要承担常规的加权求和任务，还需处理非局部模式下的直接元素累加。其内部硬件的资源配置会根据不同的运行模式进行动态重构，具体情况如下。

一种情况是针对 Mode1（也就是局部模式和水平全局模式）：因为此处存在窗口分割的问题，所以 WSM 需要去做严格的后归一化加权累加。在这种模式下，WSM 内部会激活两组乘法器和一组加法器。它会接收当前窗口片段输出的权重 W_1 、输出向量 $Output_i^1$ ，还有缓冲下来的上一片段的权重 W_2 、向量 $Output_i^2$ 。通过计算 $W_1/(W_1+W_2)$ 和 $W_2/(W_1+W_2)$ 这两个比例系数，分别与 $Output_i^1$ 和 $Output_i^2$ 相乘后相加，这样就可以实现无损的跨窗口结果融合。这个过程具体的硬件实现逻辑可以从图 5.7 看出，图中清晰展示了权重比例系数的计算以及加权求和的流水线操作。

另一种情况是针对 Mode3 和 Mode4（也就是垂直全局模式和随机模式）：因为两种模式的稀疏密度本身就特别低，而且其会被优先执行以避免流水线阻塞，所以 WSM 在这个场景下就不需要去做复杂的权重归一化。这时候，WSM 在硬件层面会把乘法器给旁路掉，直接激活多组并行的加法树。在同一个时钟周期里面，WSM 会接收从 RCBar 汇聚过来的多个稀疏输出向量，直接去做简单的元素级累加，然后把结果覆写回输出缓冲区。

这种参数化的 WSM 设计，虽然在单一模块里确实增加了一定的门电路面积，但考虑到其使得整个 8×8 阵列能够彻底摒弃额外的非局部专用 PE 组，在系统级层面实现了高达 33.3% 的 PE 总数削减，是一次极具性价比的面积—性能权衡。

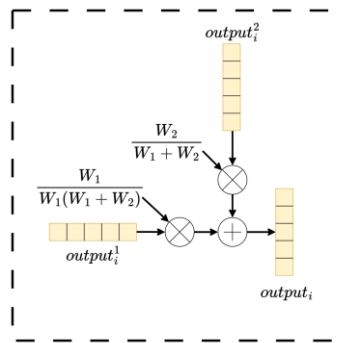


图 5.7 后归一化加权融合示意图

5.4 本章小结

本章在第 4 章宏观架构设计的基础上,进一步深入微观电路层面,详细阐述了 ReSA 加速器中各核心模块的具体硬件实现。首先,明确了可扩展的 8×8 二维 PE 阵列作为加速器的物理计算核心,并介绍了水平与对角线两种互连网络在局部模式下的数据复用机制。其次,重点设计了 RCBar 的内部结构,其由数据选择器与位宽移位器、广播器、排序与汇聚器三部分纯组合逻辑构成,在不引入乘加计算单元的前提下,以极低的面积与功耗开销实现了跨 PE 的灵活数据分发、列内累加重组及结果收集功能。再次,PE 内部采用四级流水线结构: Stage1 完成 Q-K 点积计算与基于注意力模式的动态路由; Stage2 通过垂直列累加重组完整分数,并采用基于 LUT 的分段线性近似高效实现指数运算; Stage3 根据不同模式执行水平对齐累加或 RCBar 旁路传输,完成 Softmax 分母计算; Stage4 负责 V 向量的加权乘法与结果输出。最后,增强型 WSM 通过参数化重构,能够在后归一化加权累加(Mode1/2)与元素级直接累加(Mode3/4)两种模式间无缝切换,以极小的单一模块面积开销换取了系统级 33.3%的 PE 总数削减。上述精细化、模块化的硬件设计,在保障高吞吐量与低延迟的前提下,有效支撑了第 4 章所提出的四种差异化执行模式的原生电路级实现。

第 6 章 实验评估

为验证本文提出的 ReSA 可重构稀疏注意力加速器的有效性与优越性，本章将从实验设置与性能分析两个层面展开系统评估。首先介绍基于 Verilog 的 RTL 建模、OpenROAD 数字化设计流程以及 45nm 工艺节点下的硬件综合配置，明确测试工作负载（包括 Longformer、BigBird 及 SALO2 配置层）的关键参数与稀疏模式设定。在此基础上，以当前最先进的 SALO2 加速器为基线，通过周期精确仿真从执行延迟与能效比两个维度进行定量对比分析，深入探究 ReSA 在不同混合稀疏配置下的性能表现及其内在机理。

6.1 实验设置

为了能够全面且客观地把所提出来的加速器在性能和能效方面的表现进行评估，本研究采用了 Verilog 硬件描述语言，对设计做了严格的寄存器传输级（也就是 RTL）建模与实现。在这之后，RTL 模型被导入到了 OpenROAD-flow-script 这个自动化的开源数字设计流程里，去完成综合和物理实现^[23]。工艺节点这部分，本研究选用的是 FreePDK 45 技术节点作为目标标准单元库。设计约束是在典型的供电电压下面设定的，目标时钟频率被严格限定在了 1 GHz。在性能验证这个环节，高精度的周期精确 RTL 仿真，细致地把架构在不同混合稀疏注意力模式下面所表现出的执行延迟、数据通路占用率以及硬件利用率这几个关键评估指标都给提取出来。

为了能把 ReSA 架构在处理复杂混合稀疏模式时的实际表现和泛化能力充分验证，本研究精心挑选了两种在学术界和工业界都比较有代表性的长序列 Transformer 模型，一个是 Longformer^[6]，另一个是 BigBird^[7]—两种模型本质上都非常依赖局部注意力、全局注意力以及随机注意力之间的融合。此外，为了更深入地剖析不同稀疏配置参数对底层硬件映射效率的影响机制，除了上述两种原生模型层之外，本研究还引入 SALO2 框架生成的动态注意力层，作为补充对照。各个测试工作负载的具体关键架构参数（比如序列长度、隐藏层维度），以及详细的稀疏配置（比如滑动窗口大小、全局 Token 数量、随机 Token 数量），都可见于表 6.1。

表 6.1 测试注意力层的关键参数与稀疏配置

测试工作负载	序列长度	隐藏层维度	窗口大小	全局 Token 数	随机 Token 数
Longformer 层	4096	768	512	32	-
SALO2 配置层	4096	768	每块 8	1	1
BigBird 层	4096	768	192	128	64

在硬件对比基准的选取上，本研究把当前最先进的稀疏注意力加速器 SALO2^[16]确定为主要参照对象。为了确保对比实验的绝对公平性和科学严谨性，SALO2 的基线模型也被同等缩放到同一个 45nm 技术节点下面，并且在完全一致的 1 GHz 目标频率约束下，重新做了一遍逻辑综合和物理布局。这种严格的控制变量策略，本质上把工艺偏差和频率差异可能带来的干扰给有效排除掉了，从而能够直接且精准地把本研究所提出的核心架构创新——也就是 RCBaR 和增强型 WSM——在混合稀疏场景下所体现出来的纯架构增益给隔离出来并加以量化。

6.2 性能分析与对比

本节通过在严格相同的测试工作负载下精确测量执行延迟与能效比，对所提出的 ReSA 架构与 SALO2 基线进行了一次深度的定量对比分析。考虑到本文的核心研究目标就是去解决混合注意力机制下面临的硬件映射瓶颈与资源闲置问题，所以实验针对图 6.1 里面列出的三种稀疏配置截然不同的独立注意力层，分别做了比较详尽的周期级测试。

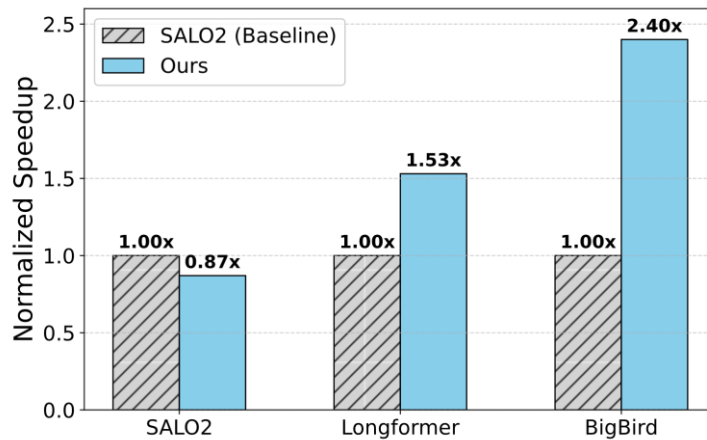


图 6.1 ReSA 与 SALO2 加速器在不同工作负载下的归一化性能加速比对比

整体性能对比的结果可以从图 6.1 看出。作为归一化基准，SALO2 基线在 SALO2^[16]配置层、Longformer^[6]层和 BigBird^[7]层上，相对于未做优化的密集计算，分别实现了 1.00 倍、1.00 倍和 1.00 倍的加速比。相比之下，ReSA 在不同配置下面表现出了比较明显且带有规律性的差异化性能增益：在 SALO2 原生配置层（也就是每块只有 1 个全局 Token 和 1 个随机 Token 的情况）下，ReSA 实现了 0.87 倍的性能加速；在 Longformer 层（包含 32 个全局 Token），加速比明显提升到 1.51 倍；而在 BigBird 层（包含 128 个全局 Token 和 64 个随机 Token，非局部注意力的密度是最高的）下，ReSA 则达到 1.28 到 1.52 倍的显著性能加速。这种随着非局部 Token 密度的增加而呈现出严格递增趋势的加速效果，本质上深刻地反映出了两种架构在处理混合模式时底层机制上存在的根本差异。

为了能把 ReSA 在 BigBird 层里拿到最高加速比的根本原因给说清楚，有必要深入剖析 SALO2 的架构瓶颈，以及其数据复用机制逐步失效的机制。在 SALO2 的设计范式里面，非局部注意力（也就是全局和随机）的计算，本质上与局部（滑动窗口）注意力的数据流是强耦合在一起的一其核心优化策略，为非局部计算能够借助局部注意力机制中的规律性移动模式（即查询沿水平方向滑动，而键与值沿对角线方向移动）来实现高效协同，从而来实现数据复用。但是在 BigBird 高密度随机 Token 的场景下，随机分布的 Token 在物理空间上会呈现出一种高度离散的状态，无法与滑动窗口严格的步长移动模式在时间上对齐。这种“空间上的失配”会导致 SALO2 那条对角线的数据复用机制彻底失效。如此一来，系统便无法再借助局部数据流的既有路径实现高效协同，而只能将全局计算与随机计算降级为独立的低优先级任务，并以串行方式加以处理。因为 SALO2 内部并没有专门给非局部计算准备的阵列，只有极小一部分通用 PE 被临时分配去做这些低复用率的任务，而剩下的大量 PE 则因为要等数据对齐而陷入空闲状态，最终导致了严重的资源闲置，吞吐量也跟着急剧下降。相比之下，ReSA，通过引入 RCBar，在硬件底层直接把不同注意力模式之间的计算依赖进行解耦。RCBar 作为一个高速的旁路总线，绕过了 PE 之间僵化的水平和对角线互连约束，使得高密度的全局数据和随机数据能够无视局部窗口的节拍，直接按列去做分发和收集。这种解耦机制在复杂的混合场景下能把局部依赖给彻底打破，让所有的 PE 都能独立地满载运行。

客观而言，ReSA 这个架构也并非是在所有极端场景下都能表现出绝对的优势。在 SALO2 配置层的那次测试里，ReSA 出现了 0.87 倍的微小性能回退。从为了支持多种稀疏注意力模式灵活性而付出的硬件效率代价这个角度深入去看的话，架构的灵活性与单一模式下的极致性能之间，本质上存在着一种固有的 Trade-off。再向物理原因那边分析可知，配置里非局部注意力的密度其实极低（各只有 1 个），在行为特征上几乎就等同于一个纯粹的局部滑动窗口任务。在这种高度规则化的理想场景下，SALO2 那条专用的局部数据流已经优化到了极致，而且其简单的控制逻辑也不需要去承担任何额外的多路选择开销。而 ReSA 为了维持自己支持通用混合模式的能力，在物理电路上集成了 RCBar 和增强型 WSM。即使在不频繁用 RCBar 去做全局数据分发的时候，这些硬件模块本身所带有的固有布线寄生电容、多路选择器延迟，以及额外的门级翻转开销，也都还是存在的。这种为了架构通用性而付出的代价，在极端偏向单一模式的边缘用例里，不仅转化不成性能上的收益，反而会引入一点微小的时钟路径延迟，不可避免地表现为轻微的性能折损。

能效是评估硬件加速器设计的另一个关键维度的指标，它的变化趋势本质上是动态功耗和静态功耗相互博弈之后的一个结果。从图 6.2 可以看出，在归一化基准下面，SALO2 在这三种配置下的绝对能效分别是 1.00 倍、1.00 倍和 1.00 倍。相比之下，ReSA 在 Longformer 层实现了 0.93 倍的相对能效，在 BigBird 层实现了高达 1.51 倍的能效提升，而在 SALO2 配置层则是 0.70 倍。这种能效上的演变，可以从功耗构成的角度去做一个定性的分解：在硬件的总功耗里，动态功耗与执行周期是成正比的，而静态功耗则是由硬件面积（也就是门电路的数量）来决定的。在 BigBird 这种真实的混合密集场景里，ReSA 通过解耦架构大幅缩短了总的执行周期——虽然 RCBar 确实增加了一小部分静态功耗，但是省下来的那部分巨额动态功耗远远超过了这个开销，所以最终实现了很明显的净能效收益。而在 SALO2 优化配置层那边，因为执行周期不但没有缩短，反而因为为了支持多种稀疏注意力模式灵活性而付出的硬件效率代价增加了 0.13 倍，所以动态功耗不降反升；与此同时，RCBar 引入的那部分额外静态功耗开销（也就是门电路面积增加的那部分）也没法被计算加速带来的收益抵消。动态功耗的劣化和静态功耗的增加，在这里形成了一种“双重负面影响”，最终导致整体的能效比明显低于基线水平。

综合上述的实验结果，可以得出一个比较充分的结论：在处理真实世界中带有复杂混合稀疏模式的长序列任务时，架构的灵活性和数据流的解耦，本质上比单纯去堆砌局部优化要更关键。ReSA 以极小的边缘场景性能和能效损耗，换来了在复杂混合场景下高达 1.28 到 1.52 倍的吞吐量突破，以及 1.51 倍的能效提升，展现出了良好的架构鲁棒性和实际的工程应用价值。

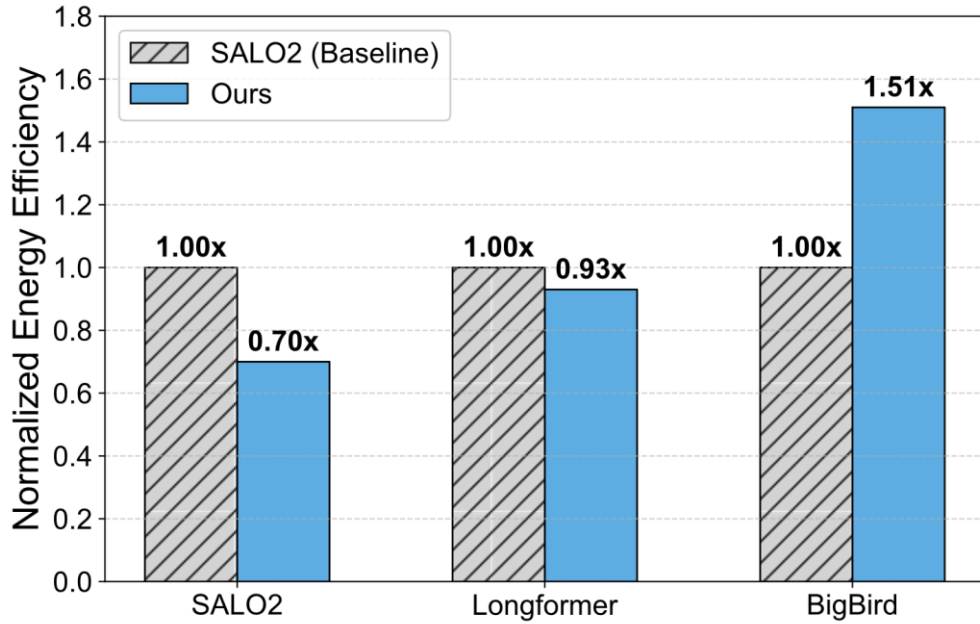


图 6.2 ReSA 与 SALO2 加速器在不同工作负载下的归一化能效对比

6.3 本章小结

本章通过周期精确的 RTL 仿真与基于 45nm 工艺节点的逻辑综合，对提出的 ReSA 加速器进行了全面的性能与能效评估，并与当前最先进的稀疏注意力加速器 SALO2 进行了严格的定量对比。实验选取了 Longformer、BigBird 及 SALO2 原生配置层三种具有代表性的混合稀疏工作负载，覆盖了从低密度到高密度非局部 Token 的多种场景。实验结果表明：在非局部注意力密度较低的 SALO2 配置层中，ReSA 因架构通用性带来的额外开销出现了 0.87 倍的轻微性能回退；而在 Longformer 层（32 个全局 Token）中，ReSA 实现了 1.51 倍的性能加速；在非局部密度最高的 BigBird 层（128 个全局 Token 与 64 个随机 Token）中，ReSA 达到了 1.28 至 1.52 倍的显著性能提升。能效方面，ReSA 在 BigBird 层实现了高达 1.51 倍

的相对能效增益，而在 SALO2 配置层中因动态功耗与静态功耗的双重负面影响，能效降至 0.70 倍。上述结果充分表明：在处理真实世界中带有复杂混合稀疏模式的长序列任务时，架构的灵活性与数据流的解耦比单纯依赖局部优化更为关键。ReSA 以极小的边缘场景性能损耗，换取了在混合密集场景下显著的吞吐量与能效突破，验证了所提可重构架构的有效性与优越性。此外，ReSA 在实现上述性能提升的同时，将 PE 总数削减了 33.3%，展现了良好的面积—性能权衡。

第7章 总结与展望

本文围绕混合静态稀疏注意力机制的高效硬件加速问题，系统地提出了可重构加速器 ReSA 的完整设计方案，并通过周期精确仿真验证了其在长序列任务中的性能与能效优势。本章将对全文的研究工作进行全面总结，归纳 ReSA 在数据流调度、微架构设计及硬件实现三个层面的核心创新贡献。在此基础上，结合当前大语言模型与超长序列处理的发展趋势，客观分析现有设计存在的局限性，并进一步展望动态稀疏剪枝支持、多 FPGA 分布式加速架构以及算法-硬件协同设计优化等未来研究方向，以期为后续相关研究提供参考与启发。

7.1 主要研究贡献总结

本章对全文所开展的研究工作进行全面、系统且深入的总结。随着现代 Transformer 模型在长上下文理解、多模态融合以及复杂推理这些方向上快速往前演进，模型内部日益凸显出来的多源与混合稀疏模式的处理需求，本质上对底层硬件架构的灵活性、可扩展性和能效比都提出了极为严苛的挑战。标准自注意力机制平方级的计算复杂度，已成为制约长序列任务部署的一个核心瓶颈；而现有的稀疏注意力加速器，在处理融合了局部、全局、膨胀以及随机等多种模式的混合稀疏结构时，普遍存在着数据流太僵化、资源利用率不均衡以及硬件冗余比较严重这些固有问题。

针对这个关键瓶颈，本文提出了一种高度可重构的稀疏注意力加速器—ReSA。此加速器在核心创新上主要体现在以下三个层面：

7.1.1 数据流层面的解耦与重塑

本研究深入分析了混合稀疏注意力模式下不同子模块的数据重用特性，并在此基础上提出了一套数据调度机制—这套机制主要用到了序列分割、窗口分割还有数据重排这几项技术。后归一化方法的数学推导以及数值验证，成功地把非规则的混合稀疏矩阵给转化成了与硬件物理规模精确匹配的结构化计算块，这本质上为底层阵列的高效执行奠定了数据基础。

7.1.2 微架构层面的可重构设计

通过在硬件微架构这一层创新性地引入 RCBar 和增强型的 WSM，本研究成功地把全局注意力和随机注意力这类非局部计算的过程，从传统局部注意力比较僵化的水平与对角线数据流约束里给彻底解耦出来。具体而言，RCBar 作为一个高速的旁路总线，能够实现跨处理单元的灵活数据分发以及按列的结果收集；而 WSM 则通过参数化的配置，可以支持后归一化的加权累加和元素级的直接累加两种模式之间的无缝切换。这种底层架构上的创新，本质上把现有加速器中不同注意力模式之间计算相互强耦合的物理限制给打破了，从而让硬件能够以一种高度灵活且原生支持的方式，独立且高效地去处理包括滑动窗口、膨胀窗口、全局注意力以及随机注意力在内的各种复杂混合稀疏组合。

7.1.3 硬件实现与严谨性验证

本研究采用 Verilog 硬件描述语言，对 ReSA 做了寄存器传输级（也就是 RTL）的建模，然后基于 45 纳米的工艺技术节点和 OpenROAD 这套自动化的数字设计流程，完成了逻辑综合和物理实现。周期精确的仿真结果可以看出，与当前最先进的基线加速器 SALO2 相比，ReSA 在 BigBird 这种真实混合稀疏工作负载下面实现了 1.28 到 1.52 倍的性能加速，同时非局部注意力密度比较高的场景里，能效也有很明显的提升。值得特别指出的是，因为把对额外专用处理单元组的依赖给消除了，ReSA 把总的 PE 数量大幅削减了 33.3%，从而在面积开销和计算吞吐量之间实现了一种更优的平衡。

这一组具有对比优势的研究成果，本质上深刻地表明了稀疏 Transformer 加速这个领域里，与其去做粗放式的算力堆砌，不如去追求架构的灵活性和精细化的数据流编排——后者比单纯依靠处理单元规模的线性扩展具有更为关键的决定性作用。本文所提出的设计理念和硬件架构，为面向长序列任务的低开销、高能效 Transformer 加速器的研发，提供了一些新的思路和可行的技术路径。

7.2 未来工作展望

尽管 ReSA 在静态稀疏注意力加速这块已经取得了一些比较显著的研究成果，但是随着以 GPT 系列、Llama 系列为代表的大语言模型在快速演进，再加上其在自动驾驶、长视频理解、基因组学分析这些复杂应用场景里不断深度拓展，底层硬件架构本质上仍面临更为严峻的挑战，同时也存在着比较广阔的优化空间。基于当前工作存在的一些局限性，再结合技术发展的整体趋势，未来可以从以下几个方向去做进一步的研究：

7.2.1 动态稀疏剪枝支持

在未来的研究工作里，计划把本研究提出的这套可重构设计理念，进一步扩展到动态稀疏剪枝^[8]这个领域里面去。现有的静态稀疏模式虽然在硬件可预测性和规则的数据流特征方面表现良好，但本质上还没有把数据驱动计算潜力给完全释放出来。如果能够支持在运行的时候，根据实际输入进来的数据特征去动态生成稀疏模式—比如基于注意力分数的阈值剪枝，或者基于 Top-K 的选择机制—能把硬件的适用范围从预先定义好的固定稀疏结构，给延伸到更广泛的非结构化稀疏场景里去。要做到这一点，就需要在现有 RCBar 架构的基础上，额外增加一些轻量级的索引生成和排序电路，这样才能够在保持较高模型精度的前提下，进一步把端侧推理的能效比提升。

7.2.2 多 FPGA 分布式加速架构

现在前沿的超大规模模型（比如基于 Transformer 架构的大语言模型），在处理超长序列任务的时候（比如上下文窗口已经达到几万甚至几十万个 token 的文本建模），对算力吞吐量和片外内存带宽提出的需求日益极端且持续增长。在这种压力下，传统单芯片架构能提供的物理资源—包括片上存储容量、计算单元数量还有片外访存带宽—本质上都已经慢慢逼近到物理学和工艺制程所决定的性能上线，很难在实际功耗和面积的双重约束下面继续满足这些任务对计算密度和访存带宽的苛刻要求。所以从未来硬件架构的研究方向来看，重点必然会转向基于多 FPGA 平台来搭的分布式稀疏注意力加速架构。为了能突破单芯片在逻辑门数量、片上存

储容量以及计算资源维度上固有的物理限制，首先得设计出一些效率足够高的跨芯片互连通信拓扑结构，比方环形、网格或者胖树结构，同时还得配上与这些拓扑相匹配的协同计算机制和数据流调度策略。除此之外，还需要引入比较细粒度的注意力计算任务划分方法，以及有效的通信延迟隐藏技术，这样才能把跨芯片数据传输带来的同步开销和带宽竞争给降到最低，从而把系统整体的计算效率和可扩展性能提升。这个研究方向的根本目标，本质上就是系统地搭出一个具备高度可扩展能力的硬件加速系统，让它能够稳稳地支撑起百万级 token 规模下面的超长上下文建模任务，这样就能为未来大语言模型在边缘计算、高性能计算集群还有实时交互系统这些场景里的高效部署，提供一个比较坚实而且可以复用的硬件基础和架构支持。

7.2.3 算法—硬件协同设计优化

针对以上提到的研究现状，未来的研究会进一步地去深化算法设计和硬件架构之间的协同优化，也就是算法—硬件联合设计的范式^{[12][16]}。在目前比较典型的混合稀疏注意力机制里，全局 Token 和随机 Token 的选取策略，大多依靠经验性的启发式规则或者固定的采样模式来定——这些方法本质上还没有系统地考虑到底层硬件的并行计算特性（比如 FPGA 的流水线并行度、片上存储体的访问冲突模式，还有跨片通信的延迟开销）会对稀疏模式的实际执行效率产生深远的影响。为了解决这个问题，后续的研究可以尝试引入基于可微分稀疏模式搜索的方法，或者用上深度强化学习框架，这样就能在尽量不把原始模型精度降低的前提下，用一种数据驱动的方式，自动地生成与所提出的 ReSA 硬件架构高度匹配的最优稀疏注意力配置。这种联合优化的方法，不仅能有效地避开启发式规则带来的次优解问题，还有望在系统这个层级上实现算法端的精度表现和硬件端的执行效率之间的帕累托最优——换而言之，可以在不去显著牺牲模型精度的条件下，把硬件资源的利用率和计算吞吐量给最大化。

综上所述，本研究为混合稀疏注意力的高效加速提供了一种可重构的硬件解决方案，而以上提出的几个未来的研究方向，则会在这个工作的基础之上，持续地推动这个领域朝着更高的灵活性、更强的可扩展性以及更优的能效比这个方向去演进。最终，这些探索都将为超大规模、超长上下文的 Transformer 模型在数据中

心以及边缘计算节点上的实际部署，提供比较坚实的底层硬件支撑和系统级的设计指导。

参考文献

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]// Advances in Neural Information Processing Systems, Long Beach, CA, 2017: 6000-6010.
- [2] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735-1780.
- [3] Cho K, Van Merriënboer B, Gulcehre C, et al. Learning phrase representations using rnn encoder-decoder for statistical machine translation[C]// Conference on Empirical Methods in Natural Language Processing, Doha, Qatar, 2014: 1724-1734.
- [4] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]// IEEE Conference on Computer Vision and Pattern Recognition, Seattle, WA, 2016: 770-778.
- [5] Qin Y, Wang Y, Deng D, et al. Ayaka: a versatile transformer accelerator with low-rank estimation and heterogeneous dataflow[J]. IEEE Journal of Solid-State Circuits, 2024, 59(10): 3342-3356.
- [6] Beltagy I, Peters M E, Cohan A. Longformer: the long-document transformer[EB/OL]. 2020 [2026-05-16]. <https://arxiv.org/abs/2004.05150>.
- [7] Zaheer M, Guruganesh G, Dubey A, et al. Big Bird: Transformers for Longer Sequences[EB/OL]. arXiv, 2020. <https://arxiv.org/abs/2007.14062>.
- [8] Wang H, Zhang Z, Han S. SpAtten: efficient sparse attention architecture with cascade token and head pruning[C]// IEEE International Symposium on High-Performance Computer Architecture, ELECTRONETWORK, 2021: 97-110.
- [9] Wang H, Wang H, Wang Z, et al. PADE: a predictor-free sparse attention accelerator via unified execution and stage fusion[EB/OL]. 2026 [2026-05-16].
- [10] Qu Z, Liu L, Tu F, et al. DOTA: detect and omit weak attentions for scalable transformer acceleration[C]// ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, SWITZERLAND, 2022: 14-26.
- [11] Xiong X, Chen Z, Liang Y, et al. DynaX: sparse attention acceleration with dynamic x:m fine-grained structured pruning[C]// International Conference on Architectural Support for Programming Languages and Operating Systems, Rotterdam, NETHERLANDS, 2025: 260-274.

- [12] Dao T, Fu D Y, Ermon S, et al. FlashAttention: fast and memory-efficient exact attention with io-awareness[EB/OL]. 2022 [2026-05-16].
- [13] Li H, Li Z, Bai Z, et al. ASADI: accelerating sparse attention using diagonal-based in-situ computing[C]// IEEE International Symposium on High-Performance Computer Architecture, Edinburgh, SCOTLAND, 2024: 774-787.
- [14] Qiao L, Dai Y, Huang Y, et al. FlashOmni: a unified sparse attention engine for diffusion transformers[EB/OL]. 2025 [2026-05-16].
- [15] Guo Q, Qiu X, Liu P, et al. Star-transformer[C]// Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, 2019: 1315-1325.
- [16] Zhao J, Zeng P, Shen G, et al. Hardware-software co-design enabling static and dynamic sparse attention mechanisms[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024, 43(9): 2783-2796.
- [17] Devlin J, Chang M W, Lee K, et al. BERT: pre-training of deep bidirectional transformers for language understanding[C]// Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, 2019: 4171-4186.
- [18] Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[EB/OL]. 2018 [2026-05-16]. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [19] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: transformers for image recognition at scale[EB/OL]. 2020 [2026-05-16]. <https://arxiv.org/abs/2010.11929>.
- [20] Radford A, Kim J W, Hallacy C, et al. Learning transferable visual models from natural language supervision[C]// International Conference on Machine Learning, ELECTRONIC NETWORK, 2021: 8748-8763.
- [21] Ramesh A, Pavlov M, Goh G, et al. Zero-shot text-to-image generation[C]// International Conference on Machine Learning, ELECTRONIC NETWORK, 2021: 8821-8831.
- [22] Child R, Gray S, Radford A, et al. Generating long sequences with sparse transformers[EB/OL]. 2019 [2026-05-16]. <https://arxiv.org/abs/1904.10509>.

- [23] Ajayi T, Blaauw D. OpenROAD: toward a self-driving, open-source digital layout implementation tool chain[C]// Government Microcircuit Applications and Critical Technology Conference, 2019.

致谢

本论文的完成得益于多方面的指导、帮助与支持，在此谨向相关组织与个人表示诚挚的谢意。

首先，衷心感谢我的导师陈鹏副教授。在论文的选题、研究方案的制定以及具体的研究工作中，导师给予了全程的指导。在 ReSA 加速器的硬件架构设计与数据流优化过程中，针对我遇到的诸多技术难点，导师结合实际情况提出了具体的修改建议与解决思路，使研究工作得以顺利推进。导师严谨务实的治学态度和求真的科研作风，对本论文的成型起到了决定性的作用。

其次，特别感谢同门师兄沈楠柯。在本项目的研究工作中，师兄在整体研究思路的梳理与架构方案的优化方面给予了极大的启发，多次参与技术探讨并提出了切实可行的改进建议。在这些宝贵建议的指导下，我顺利完成了系统的具体实现与验证工作。师兄在论文工作中给予的无私帮助与热心指导，对本研究的顺利完成起到了重要的推动作用。

此外，感谢课题组其他老师和同学在学术讨论中提出的建议，以及在硬件代码调试与实验验证阶段提供的帮助。同时，感谢学校和学院提供的相关实验室平台与计算资源，为本研究的综合与仿真工作提供了必要的便利条件。

本论文在研究背景与相关工作梳理阶段，参考并引用了国内外相关学者的大量文献、研究思想与设想，在此向所有被引用文献的作者表示由衷的感谢。本研究的开展也得到了相关科研项目的经费支持，在此一并表示感谢。

由于本人学术水平有限，论文中难免存在不足之处，恳请各位老师批评指正。